基于 PSO 优化的最长遮蔽时间单目标规划模型

摘要

烟幕干扰弹通过快速形成遮蔽云团有效干扰敌方侦察与制导系统，在战场伪装与防御中发挥着关键作用。针对烟幕干扰弹的投放策略问题，本文建立混合整数规划模型，运用双重退火算法、惯性权重粒子群优化算法等方法进行求解，兼顾全局搜索与局部优化，制定高质量投放策略。

针对问题一，首先分别计算导弹、烟幕干扰弹和云团中心点的运动轨迹，在烟幕弹有效时长内，计算真目标表面各点和导弹位置的连线距离云团中心是否大于烟幕有效半径，以检验导弹对真目标的视野是否完全被云团遮挡。为简便计算，将云团有效时间切割为足够小的Δt进行计算，并选取圆柱体上与导弹位置相对应的九个关键点代替整个圆柱体的表面点进行检验。最终求出烟幕干扰弹对 M1 的有效遮蔽时长为 1.391s。

针对问题二，在问题一的基础上，将无人机飞行的速度、方向，烟幕干扰弹的投掷时机、引爆时间改为决策变量，以真目标被遮蔽时间尽可能长为目标，在添加约束条件后建立规划模型，利用双重退火算法进行求解，最终求出最佳参数为：无人机速度 104.178m/s，投放时间 0.298s，起爆时间 0.568 s，航向角 0.113rad，此时最大有效遮蔽时间为 4.58 秒。

针对问题三和问题四，由于无人机数或投掷烟幕弹数的增加，在问题二的基础上额外设定参数，在问题三中设置三枚烟幕干扰弹的投放时间和引爆时间、无人机飞行方向、无人机飞行速度等 8 个参数，在问题四中设置三架无人机的运动方向、运动速度以及三枚烟幕干扰弹的投放时间和起爆时间等 12 个参数。为避免有效遮蔽时间的重复计算，将多个有效遮蔽时间序列整合成一个时间序列后去除重复的有效遮蔽时间点，同样以真目标被遮蔽的时间尽可能长为目标建立规划模型，选取惯性权重粒子群优化算法进行求解，得出最佳参数组合。第三问的情况下最佳有效遮蔽时长为 6.38s，第四问的情况下最佳有效屏蔽时长为 8.45s。

针对问题五，将投放策略的参数个数增加到 40 个，用来代表不同方案的烟幕弹投放策略。由于该问题中有多个导弹，定义全局有效遮蔽时间为有一个或多个导弹的视野被有效遮蔽的时间段，且在某一时刻视野被遮蔽的导弹数量越多投放方案的效果越好，由此设定目标函数为最大的全局有效遮蔽时间。为避免多个云团对一个导弹的有效遮蔽时间叠加，对单个导弹的有效遮蔽时间序列进行整合，再将多个导弹的遮蔽时间相加作为全局有效遮蔽时间。选取惯性权重粒子群优化算法进行求解得出最佳参数组合，此时全局遮蔽时长为 33.01s。

关键词：单目标规划、双重退火、惯性权重粒子群优化、关键点替代、多时间序列融合

一、 问题重述

1.1 问题背景

烟幕干扰作为一种高效对抗高价值制导武器的手段，因其响应迅速、成本低廉而得到广泛应用。烟幕干扰弹通过燃烧或爆炸在指定区域形成烟幕或气溶胶云团，从而干扰敌方导弹。目前相关技术的发展已能够实现烟幕干扰弹的多方式、定点化和精准化投放。该技术在国防领域具有重要意义，尤其是在战场伪装与防御中发挥着关键作用。通过无人机投放烟幕干扰弹可快速到达指定区域，形成多层级遮蔽效果，实现空中快速部署，覆盖度广且灵活性强。

1.2 基本问题

问题 1：题目给出 FY1 以 120m/s 的速度朝向假目标方向飞行，受领任务 1.5 s 后投放 1 枚烟幕干扰弹，间隔 3.6 s 后起爆。根据以上信息需要求出无人机 FY1 投放 1 枚烟幕干扰弹 M1 的有效遮蔽时长。

问题 2：在确保无人机 FY1 投放 1 枚烟幕干扰弹对 M1 的遮蔽时间尽可能长的目标下，需确定无人机 FY1 的飞行方向、飞行速度、烟幕干扰弹的投放位置及起爆点。

问题 3：设计合理的烟幕干扰弹投放策略，使得在无人机 FY1 投放 3 枚干扰弹对 M1 进行干扰时，对$M1$ 的遮蔽时间尽可能长，并将设计结果记录在表格中。

问题 4：设计烟幕干扰弹的投放方案，使得 3 架无人机 FY1、FY2、FY3 分别投放 1 枚烟幕干扰弹时对 M1 的有效干扰时间尽可能长，并将结果填入表格中。

问题 5：要求使用 5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，对 M1、M2、M3 等 3 枚导弹进行干扰，根据此项任务设置烟幕干扰弹的投放策略，使得导弹视野被遮蔽的时间尽可能长，并将方案填写在表格中。

二、 问题分析

本题需在给定条件下，设计无人机投放烟幕干扰弹的策略；题目中各物体三维坐标与导弹的速度与方向都已经给出，在无人机初始方向、初始速度、投弹时间、引爆时间、投弹数量、无人机数量、导弹数量等参数发生变动时，求解使得真目标在导弹的视野中被烟幕遮蔽时间尽可能长的最优化问题，为混合整数规划问题。问题一至问题五循序渐进，逐渐增加新的变量，最后引导至问题五中，五架无人机投放至多三枚烟雾弹使得烟幕遮蔽时间最长的最优化问题。

2.1 问题一的分析

问题一要求我们在给定单架无人机初始方向、速度和烟幕干扰弹投放时间、起爆时间等参数的情况下，求解投掷单枚烟幕干扰弹对于导弹 M1 的遮蔽时间。整个运动系统的信息明确，可以直接对结果进行计算。首先求出导弹、无人机的运动方向向量以及其运动特定时间后的坐标位置；随后分析烟幕干扰弹掷出后的轨迹以及爆炸产生的云团中心点的坐标；最后在烟幕弹有效时长内，通过计算真目标表面各点和导弹位置的连线距

离云团中心是否大于烟幕有效半径，以检验导弹对真目标的视野是否完全被云团遮挡，同时计算遮蔽时间。为简便计算，将云团有效时间切割为足够小的ΔT进行运动位置的计算，并选取圆柱体上与导弹位置相对的九个关键点代替整个圆柱体的表面点进行检验。

2.2 问题二的分析

该问题在问题一的基础上，将无人机飞行的速度、方向，烟幕干扰弹的投掷时机、引爆时间改为决策变量，分析烟幕干扰弹掷出后的轨迹以及爆炸产生的云团中心点的坐标，使得真目标被遮蔽时间尽可能长。此为数学优化问题，需要合理设立目标函数和约束条件，利用优化算法对最佳结果进行求解。

2.3 问题三的分析

此问题中，$FY1$ 可投掷三枚烟幕干扰弹对$M1$ 进行遮挡，要求求解使得遮蔽时间最长的方案。由于投放烟幕干扰弹的数量增加，需要在问题二的基础上额外设定参数，在该问题中设置三枚烟幕干扰弹的投放时间和引爆时间、无人机$FY1$ 飞行方向和无人机$FY1$ 飞行速度等 8 个参数，并合理设定目标函数和约束条件，选取优化算法进行求解。

2.4 问题四的分析

在问题四中，有三架无人机各投掷一枚烟幕干扰弹对导弹$M1$ 进行遮挡，要求求解使得遮蔽时间最长的方案。由于无人机数量的增加，需要在问题二的基础上设置额外的参数，在该问题中设置三架无人机的运动方向、运动速度以及三枚烟幕干扰弹的投放时间和起爆时间等 12 个参数，并合理设定目标函数和约束条件，选取优化算法进行求解。

2.5 问题五的分析

在问题五中，系统中有三架无人机分别投掷五枚烟幕干扰弹，对三枚导弹进行遮挡。在前几问的基础上，将投放策略的参数个数增加到 40 个。由于该问题中有多个导弹，可以认为某一时刻对单个导弹视野的遮蔽也是有效的，且在某一时刻视野被遮蔽的导弹数量越多投放方案的效果越好。则需在不同时刻对系统中不同导弹的视野分别进行遮蔽检验，有效时间为三个导弹视野被遮蔽的时间的总和。在合理设定目标函数和约束条件后，选用优化算法进行求解。

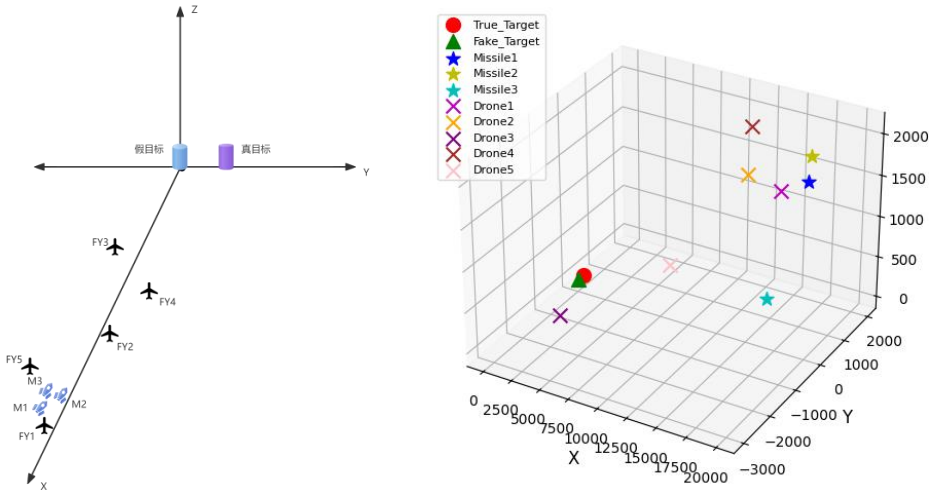

图 1 各物体位置示意图

三、 模型假设

（1）在导弹的视野中，若在圆柱体上选取的九个关键点被云团遮蔽，则可认为整个圆柱体被遮蔽。

（2）在导弹的视野中，圆柱体上底面和下底面的关键点与中心点的距离同等，其误差可以忽略不计。

（3）有多个导弹时，某一时刻对单个导弹视野的遮蔽看作是有效遮蔽时间。

（4）有多个导弹时，某一时刻视野被遮蔽的导弹数量多的方案由于视野被遮蔽的导弹数量少的方案。

（5）有多个导弹时，整个方案的有效时间为每个导弹的有效时间之和。

四、 符号说明

表 1：变量定义表

符号	含义	符号	含义
$True$	真目标位置	B_j^k	第 j 架无人机第 k 枚烟幕干扰弹的起爆时间
F	假目标位置	T_j^k	第 j 架无人机投放第 k 枚烟幕干扰弹引爆后经过的时间
M_i	第 i 枚导弹的初始位置	$key_{i,t}^l$	第i枚导弹在t时刻视野中真目标的第l关键点
$M_{i,t}$	第i枚导弹在t时刻的位置	g	重力系数
FY_j	第 j 架无人机的初始位置	r	真目标底面半径
V_0	导弹速度	R	烟幕干扰弹有效范围的半径
φ_j	第 j 架无人机的方向	h	真目标的高
V_j	第 j 架无人机的速度	$n_{j,t}^k$	第 j 架无人机投放的第 k 枚烟幕干扰弹引爆后 t 时刻的中心位置
A_j^k	第 j 架无人机第 k 枚烟幕干扰弹的投放时间		

五、 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 问题一模型的建立

（1）求导弹坐标

已知无人机$FY1$ 坐标为$FY1$: $(17800,0,1800)$、真目标底面圆心坐标T: $(0,200,0)$、假目标坐标F: $(0,0,0)$、导弹$M1$ 起始坐标$M_{1,0}$: $(20000,0,2000)$，如图所示：

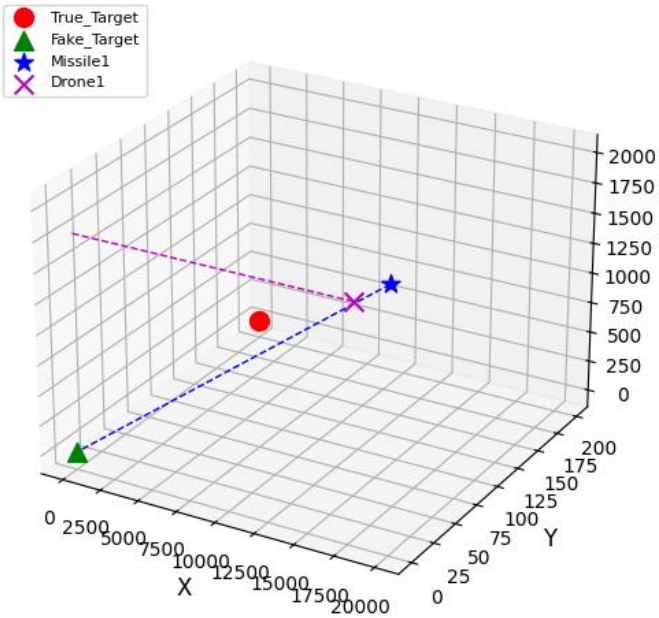

图 2 问题一位置示意图

由此可以计算导弹$M1$ 的飞行方向：

$$\mu_1 = \frac{F-M1}{|F-M1|} \tag{1}$$

以及t时刻导弹$M1$ 的坐标$M_{1,t}$:

$$M_{1,t} = M_{1,0} + \mu_1 * v_0 * t \tag{2}$$

（2）求云团中心点坐标

在该问题中，我们已知无人机$FY1$ 的受领任务后投放烟幕干扰弹的间隔时间$A_1^1 = 1.5s$，投放时间与起爆时间的间隔$B_1^1 = 3.6s$，飞行速度v_1为 120m/s，方向为朝着假目标的方向，即

$$\varphi_1 = \frac{F-FY1}{|F-FY1|} \tag{3}$$

再设烟幕干扰弹从起爆到失效的时间为T_1^1，由于云团中心 10m 范围内的烟雾浓度有效时间为 20s，则$T_1^1 \in [0,20]$。由此可以计算出，在t时刻云团中心的坐标：

$$n_{1,t}^1 = FY1 + \left(A_1^1 + B_1^1\right) * \varphi_1 * v_1 - \frac{1}{2} * g^2 * (B_1^1)^2 * Z - 3 * T_1^1 * Y \tag{4}$$

其中，$Z = (0,0,1)$，此时有

$$t = A_1^1 + B_1^1 + T_1^1 \tag{5}$$

（3）以关键点代替整个真目标圆柱体

对于真目标圆柱体上的每一点，我们可以表示为：

$$P = True + h \cdot Z + r \cdot (\cos\theta \cdot X + \sin\theta \cdot Y)，\theta \epsilon [0,2\pi] \tag{6}$$

其中，h为圆柱体的高度，r为圆柱体的底面半径，$X = (1,0,0)$，$Y = (0,1,0)$。

为简便计算，减小计算量，选取圆柱体的 9 个关键点代替圆柱体的全体表面点，这 9 个关键点为以导弹位置，真目标底面圆心，顶面圆心三点连成的面与圆柱体上下面圆周的交点（除去底面后方无法被看见的点），以及与该面垂直且过圆柱体中轴线的面与圆柱体上下圆周的四个交点，加上上下两交点连线的中点。

为求这九个点的坐标，设t时刻第i导弹与真目标之间的方向向量为ω_1^t，则

$$\omega_{i,t} = \frac{F - M_{i,t}}{|F - M_{i,t}|} \tag{7}$$

将该向量投影到圆柱体底面上，并变换为单位向量，若$\omega_i = (x_i, y_i, z_i)$，则变换后的方向向量$\omega_{i,t}^{'} = (x_i, y_i, 0)/||\omega_{i,t}^{'}||$，在底面上，与$\omega_{1,t}^{'}$垂直的向量为$\omega_{1,t}^{''} = (-y_i, x_i, 0)$.

于是，九个关键点的坐标可如下表示：

$$key_{i,t}^{1} = True - \omega_{i,t}^{'} * r \tag{8}$$

$$key_{i,t}^{2\ or\ 3} = True \pm \omega_{i,t}^{''} * r \tag{9}$$

$$key_{i,t}^{4\ or\ 5} = key_{i,t}^{2\ or\ 3} + (h * Z)/2 \tag{10}$$

$$key_{i,t}^{6\ or\ 7} = True + h * Z \pm \omega_{1,t}^{'} * r \tag{11}$$

$$key_{i,t}^{8\ or\ 9} = True + h * Z \pm \omega_{1,t}^{''} * r \tag{12}$$

其结构如图所示：

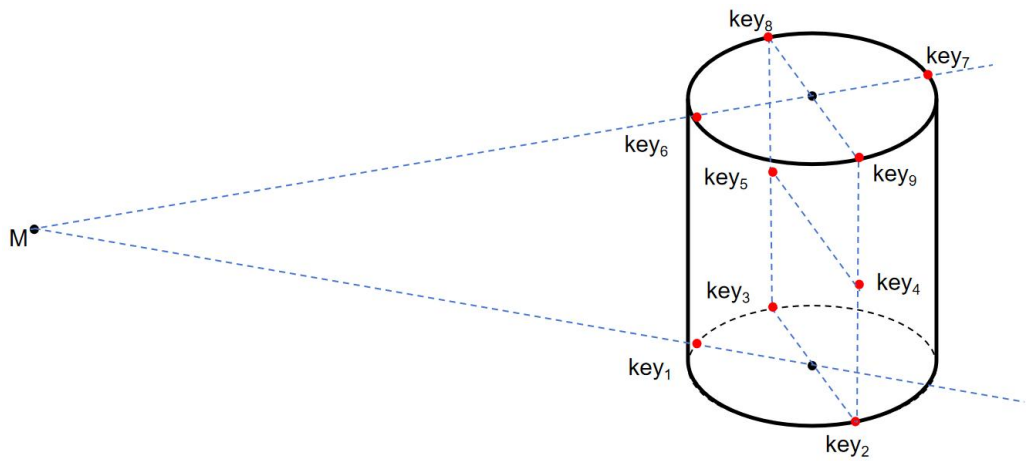

图 3 圆柱体 9 个关键点示意图

（4）检验真目标是否被云团完全遮蔽

若要检验云团是否遮蔽圆柱体，仅需检验这九个点是否完全被云团检验，即检验这九个点与导弹位置的连线与云团中心点坐标的最短距离是否小于云团半径。

在t时刻，第l关键检查点与导弹位置的连线的最小距离由投影得到：

$$s_{1,t}^{l}=\frac{(n_{1,t}^{1}-key_{1,t}^{l})\cdot(M_{1,t}-key_{1,t}^{l})}{\left||M_{1,t}-key_{l,t}|\right|^{2}} \tag{13}$$

$$s_{1,t}^{l'}=\min\{1,\max\{0,s_{1,t}^{l}\}\} \tag{14}$$

其中$key_{1,t}^{l}$为t时刻的第l关键点坐标，$l\in[1,9]$，则连线与云团中心点的最小距离为：

$$\min L_{1,t}^{l}=||key_{1,t}^{l}+s_{t}^{l'}*\left(M_{1,t}-key_{1,t}^{l}\right)-n_{1,t}^{1}|| \tag{15}$$

需要满足该距离小于云团的有效半径，即：

$$\left(\min L_{1,t}^{l}\right)^{2}\leq R^{2} \tag{16}$$

其中，R为云团有效半径距离。

（5）计算求解

将T_1^1分割成足够小的时间段ΔT，检验在T_1^1范围内真目标被云团完全遮蔽的情况，则检验ΔT 的左端点即可判断ΔT 范围内真目标是否被云团完全遮蔽。令$\Delta T(1)$为通过检验的时间，则$\Delta T(2)$为未通过检验的时间，通过计算，可以得到云团有效的遮蔽时间。

5.1.2 问题一模型的求解

使用 Python 代码模拟上述模型可得有限屏蔽时间为 1.39199s，以烟幕干扰弹起爆时间为 0 时刻，有效屏蔽时间在T_1^1范围内的分布如下图：

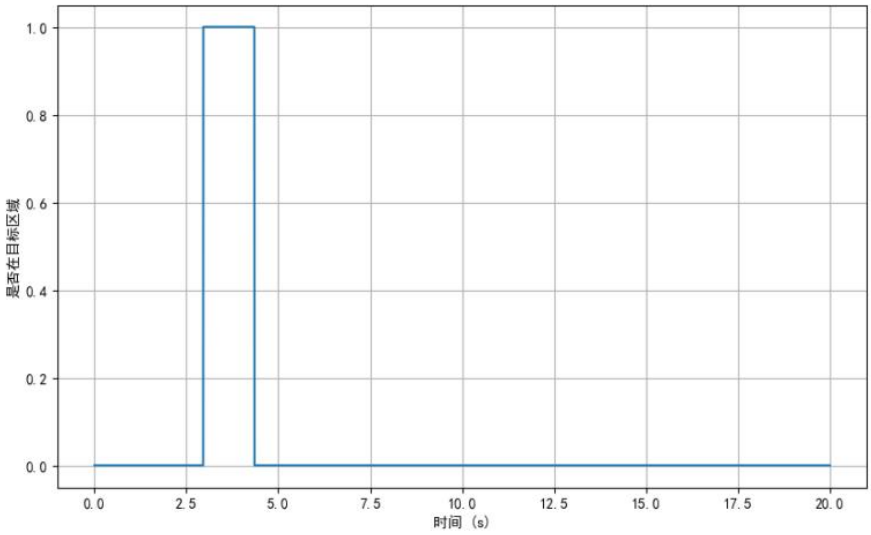

图 4 有效屏蔽时间分布示意图

有效屏蔽时间内某一时刻系统状态如下图：

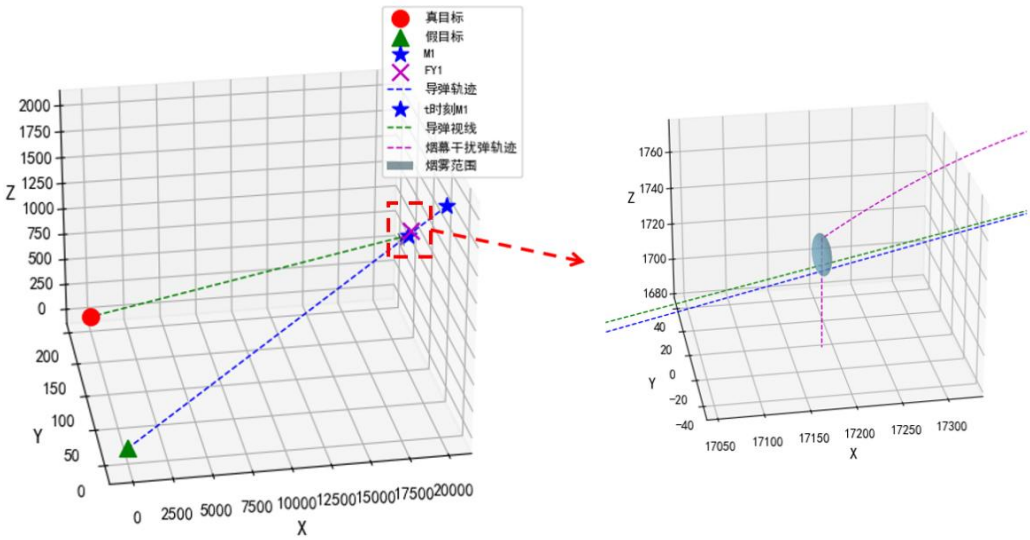

图 5 有效屏蔽时间内某一时刻系统状态示意图

5.2 问题二模型的建立与求解

5.2.1 问题二模型的建立

（1）设定参数

在问题一的基础上，重新设置决策变量θ，$\theta \in [0, 2\pi]$，则无人机飞行方向向量则可表示为$\varphi_1 = [cos(\theta), \sin(\theta), 0]$，另设无人机飞行速度$v_1$，无人机$FY1$的受领任务后投放烟幕干扰弹的间隔时间$A_1^1$，投放时间与起爆时间的间隔$B_1^1$，则此时在$t$时刻云团中心的坐标为：

$$n_{1,t}^1 = FY1 + \left(A_1^1 + B_1^1\right) * \varphi_1 * v_1 - \frac{1}{2} * g^2 * (B_1^1)^2 * Z - 3 * T_1^1 * Y \tag{17}$$

其中：

$$t = A_1^1 + B_1^1 + T_1^1 \tag{18}$$

（2）目标函数

为使真目标的遮蔽时间最长，设定目标函数：

$$\text{Max sumT} = \int_0^{20} f\left(T_1^1\right) dT_1^1 \tag{19}$$

其中，$f(t)$为检验此时真目标是否被遮蔽的函数，即

$$f\left(T_1^1\right) = \begin{cases} 1, & \left(\min L_{1,t}^1\right)^2 \le R^2 \\ 0, & \left(\min L_{1,t}^1\right)^2 > R^2 \end{cases} \tag{20}$$

$$\min L_{1,t}^1 = ||key_{1,t}^1 + s_{1,t}^{1'} * \left(M_{1,t} - key_{1,t}^1\right) - n_{1,t}^1|| \tag{21}$$

$$s_{1,t}^{1'} = \min\{1, \max\{0, s_{1,t}^1\}\} \tag{22}$$

$$s_{1,t}^{l}=\frac{(n_{1,t}^{1}-key_{1,t}^{l})\cdot(M_{1,t}-key_{1,t}^{l})}{\left\|M_{1,t}-key_{l,t}\right\|^{2}} \tag{23}$$

（3）约束条件

根据题目要求，模型中隐含如下约束条件：

①保证投弹与引爆在导弹抵达假目标之前：

$$t \leq \frac{|F-M1|}{300}$$

②保证无人机在受领任务之后再投弹：

$$A_1^1 \geq 0$$

③保证烟幕干扰弹在投弹之后再起爆：

$$B_1^1 \geq 0$$

④云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内可为目标提供有效遮蔽：

$$T_1^1 \in [0,20)$$

⑤无人机飞行速度介于 70-140m/s 之间：

$$v_1 \in [70，140]$$

（4）模型总括

$$\text{Max sum_T}=\int_0^{20} f(T_1^1)dT_1^1$$

$$\text{s.t.}\begin{cases} 0<t\leq\dfrac{F-M1}{300} \\ v_1\in[70,140] \\ t=A_1^1+B_1^1+T_1^1 \\ T_1^1\in[0,20) \\ 0\leq A_1^1,B_1^1\leq t \\ \theta\in[0，2\pi] \end{cases}$$

（5）利用双重退火算法求解

双重退火算法是一种基于概率的全局优化算法，通过在外层退火和内层退火中协同工作，双重退火在模拟退火算法基础上增加了两层退火机制，使得算法在全局搜索和局部优化之间达到平衡。固体在高温状态时，分子含有较大内能，分子会进行剧烈的无规则运动，稳定性差，随着高温的物体温度逐步下降，分子所含有的内能逐步降低，分子的稳定性逐步提高，最终固体达到常温，分子的状态也趋于稳定，内能达到最小值。

双重退火算法则模拟这一过程，双重退火算法包含两个部分：Metropolis 算法和退火过程，分别对应了退火的内循环和外循环。在退火过程的外循环中，需先设置较高的初始温度，然后使温度参照降温系数按比例下降，当温度达到最终温度时退火过程结束。

在 Metropolis 算法的内循环中，在外层温度固定的前提下，内层温度用于局部优化，通过在当前区域内多次迭代，进一步搜索更优解，提高搜索精度和跳出局部最优的概率。

在模拟退火算法中，先设置一较高的初始温度并随机生成一个初始解，在原有解的邻域内随机产生一个新解，比较原有解与新解的目标函数值，根据两个解的目标函数值和环境温度值得出一个通过概率，以这一概率与随机数 ε 对比决定是否接受新解。在接受新解后，将新解作为原有解，重复上述步骤。在这一过程中，同步参照初始设定的降温比例降低环境温度，随着温度的降低，新解的通过概率也会逐步降低，在温度足够低时（即达到设定的终止温度时），新解的通过概率趋近于 0，最终稳定下来，当目标函数越大时，产出最优解。双重退火算法在此基础上扩展为两个退火阶段：外层退火负责全局探索，内层退火在当前解邻域内进行精细局部优化，从而提高跳出局部最优的能力和全局收敛性能。

新解的通过概率具体计算方式如下：

$$P = \begin{cases} 1, & f(n+1) > f(n) \\ e^{\frac{f(n+1)-f(n)}{T}}, & f(n+1) \le f(n) \end{cases} \tag{24}$$

若新解优于原有解，则新解被接受的概率为 1，若新解劣于原有解，则将通过上式所得的概率与一在区间 [0,1] 上产生的均匀分布的随机数 ε 进行比较，若 $\varepsilon < P$，则接受新解，否则保持原有解。可见，在环境温度越低时，新解的通过概率越低，最终的最优结果会趋于稳定。

接受新解的三条原则如下，在双重退火中，这些原则需分别应用于外层和内层退火：

I. 设置外层

II. 随着温度的下降，接受使目标函数上升的解的概率要逐渐减小；

III. 当温度趋于零时，只能接受目标函数下降的解。

模拟退火算法的具体步骤是：

Step1: 设置外层初始温度 T_1、内层初始温度 T_2、降温系数α_1和α_2，以及终止温度和各层迭代次数L_1 和L_2；

Step2: 随机产生初始解 S_0；

Step3: 外层循环开始，固定外层温度 T_1，进入内层退火；

Step4: 内层循环中，随机生成新解并与当前解比较，根据 Metropolis 准则决定是否接受；

Step5: 完成 L_2 次迭代后，降低内层温度 $T_2 = \alpha_2 * T_2$，重复步骤四直到 $T_2 \le T_{2\,min}$；

Step6: 内层优化完成后，降低外层温度 $T_1 = \alpha_1 * T_1$

Step7: 重复步骤三至步骤六，直到 $T_1 \le T_{1\,min}$；

Step8: 得出全局最优解。

5.2.2 问题二模型的求解

使用双重退火算法对规划模型进行求解后，得出的最佳参数如下：

表 2：问题二变量最佳参数值

变量	最佳参数值
无人机速度v_1	104.17814618028933 m/s
投放时间A_1^1	0.2982345129442978 s,
起爆时间B_1^1	0.5681990450982382 s
航向角θ	0.11285202932410243 rad

此时有效遮蔽时间最大，其数值为 4.58 秒。有效屏蔽时间在云团有效时间T_1^1内的分布状况如下：

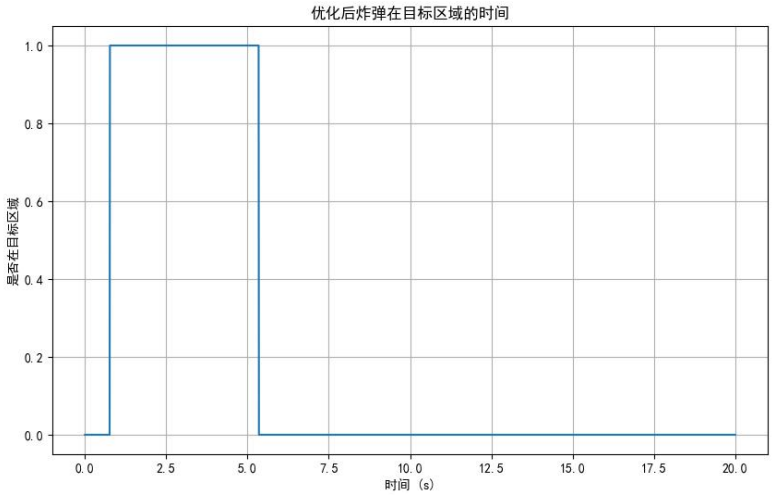

图 6 有效屏蔽时间在云团有效时间内的分布状况示意图

在有效屏蔽时间内，某一时刻整个系统的状态如下图所示：

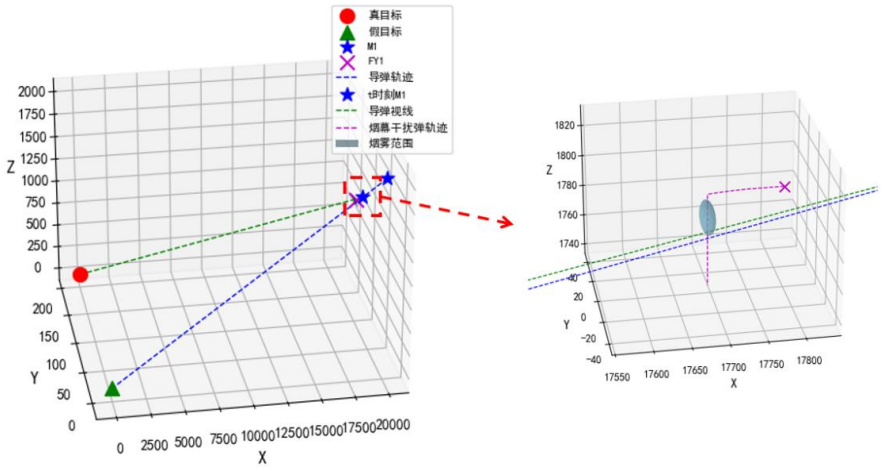

图 7 有效屏蔽时间内某一时刻整个系统的状态示意图

5.3 问题三模型的建立与求解

5.3.1 问题三模型的建立

由于该问题涉及多枚烟幕干扰弹的投放方案，为避免多个云团对同一枚导弹的遮蔽时间被重复计算，将整个运动系统从无人机受领任务开始计时，记作$t=0$，导弹抵达假目标的时刻认为是最终时刻，将整个运动过程分割为颗粒度为 0.001s的时间序列，将不同烟幕干扰弹的有效时间同一整合到该序列上。此外，若在运动系统的某一时刻，多个云团同时遮蔽了导弹的视野，不重复计算有效遮蔽时间，即序列的取值范围为{0,1}.

且此时为较高纬度的数学优化问题，采用 PSO 算法对模型进行求解。

（1）设定参数

与前两问不同，在第三问中，$FY1$ 可以投放三枚烟幕干扰弹对导弹$M1$ 进行遮蔽，因此我们需要额外设定参数。设第一枚烟幕弹的投放时间为A_1^1，引爆时间为B_1^1，第二枚烟幕弹的投放时间为A_1^2，引爆时间B_1^2，第三枚烟幕弹的投放时间为A_1^3，引爆时间为B_1^3。同样设置其余变量，即决策变量θ、无人机飞行速度v_1。在t时刻，第k枚烟幕干扰弹的云团中心位置为：

$$\mathrm{n}_{1,t}^{k}=\mathrm{FY1}+\left(\mathrm{A}_1^{k}+\mathrm{B}_1^{k}\right)*\varphi_1*\mathrm{v}_1-\frac{1}{2}*\mathrm{g}^2*(\mathrm{B}_1^{k})^2*\mathrm{Z}-3*\mathrm{T}_1^{k}*\mathrm{Y} \tag{25}$$

其中：

$$\mathrm{t}^{k}=\mathrm{A}_1^{k}+\mathrm{B}_1^{k}+\mathrm{T}_1^{k} \tag{26}$$

（2）目标函数

在有多枚烟幕干扰弹的情况下，重新设定目标函数：

$$\mathrm{Max}\ \sum_{k=1}^{3}\mathrm{sumT}_1^{k}=\sum_{k=1}^{3}\int_0^{20}\mathrm{f}\left(\mathrm{T}_1^{k}\right)\mathrm{dT}_1^{k} \tag{27}$$

其中，$f(t)$为检验此时真目标是否被遮蔽的函数，即

$$\mathrm{f}\left(\mathrm{T}_1^{k}\right)=\begin{cases}1, & \left(\mathrm{minL}_{1,t}^{k,l}\right)^2\le \mathrm{R}^2\\ 0, & \left(\mathrm{minL}_{1,t}^{k,l}\right)^2>\mathrm{R}^2\end{cases},\ \mathrm{k}=1,2,3 \tag{28}$$

$$\mathrm{minL}_{1,t}^{k,l}=||\mathrm{key}_{1,t}^{1,l}+\mathrm{s}_{1,t}^{k,l'}*\left(\mathrm{M}_{1,t}-\mathrm{key}_{1,t}^{1,l}\right)-\mathrm{n}_{1,t}^{k}|| \tag{29}$$

$$\mathrm{s}_{1,t}^{k,l'}=\min\left\{1,\max\left\{0,\mathrm{s}_{1,t}^{k,l}\right\}\right\} \tag{30}$$

$$\mathrm{s}_{1,t}^{k,l}=\frac{(\mathrm{n}_{1,t}^{k}-\mathrm{key}_{1,t}^{1,l})\cdot(\mathrm{M}_{1,t}-\mathrm{key}_{1,t}^{1,l})}{\left\|\mathrm{M}_{1,t}-\mathrm{key}_{1,t}^{1,l}\right\|^2} \tag{31}$$

（3）约束函数

根据题目要求，模型中隐含如下约束条件：

①保证投弹与引爆在导弹抵达假目标之前：

$$t \le \frac{|F-M1|}{300}$$

②保证无人机在受领任务之后再投弹：

$$A_1^k \ge 0,\ k=1,2,3$$

③保证烟幕干扰弹在投弹之后再起爆：

$$B_1^k \ge 0,\ k=1,2,3$$

④云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内可为目标提供有效遮蔽：

$$T_1^k \in [0,20),\ k=1,2,3$$

⑤无人机飞行速度介于 70-140m/s 之间：

$$v_1 \in [70，140]$$

⑥第一枚、第二枚、第三枚烟幕干扰弹的投放有先后顺序，且两枚导弹投放间隔超过 1s：

$$A_1^1+1<A_1^2$$
$$A_1^2+1<A_1^3$$

（4）模型总括

$$\text{Max}\sum_{k=1}^{3}\text{sumT}_1^k=\sum_{k=1}^{3}\int_0^{20}f\left(T_1^k\right)dT_1^k$$

$$s.t.\begin{cases} 0<t^k\le\dfrac{F-M1}{300} \\ v_1\in[70,140] \\ t^k=A_1^k+B_1^k+T_1^k \\ T_1^k\in[0,20) \\ 0\le A_1^k,B_1^k\le t \\ \theta\in[0，2\pi] \\ A_1^1+1<A_1^2 \\ A_1^2+1<A_1^3 \end{cases}$$

（5）利用惯性权重 PSO 求解

惯性权重粒子群优化算法是在标准粒子群优化算法的基础上引入惯性权重因子ω而提出的改进算法，目的是在全局搜索与局部搜索之间取得平衡。当ω较大时，粒子保留更多原有速度，使得粒子在搜索过程中能够探索更大范围，增强其全局搜索能力，当ω较小时，粒子速度衰减较快，增强局部搜索能力。

在更新粒子速度时，惯性权重控制了粒子保留原有速度的程度，提升粒子全局搜索能力，又能集中于最优区域进行局部优化。初始阶段惯性权重较大，粒子能够跳出局部最优，扩大搜索范围。后期惯性权重逐渐减小，粒子速度下降，使群体逐步集中于最优区域，加速收敛。惯性权重 PSO 参数控制简便、计算效率高，只需引入一个惯性权重参数，无需大幅增加计算量，适合大规模优化问题，且对非线性、复杂优化等问题表现

良好，适用于连续优化和离散优化。

5.3.2 问题三模型的求解

使用 PSO 算法对规划模型进行求解后，得出的最佳参数如下：

表 3：问题三变量最佳参数值

变量	最佳参数值
第一枚烟幕弹的投放时间A_1^1	0.01 s
第一枚烟幕弹引爆时间B_1^1	0.01 s
第二枚烟幕弹的投放时间A_1^2	1.01 s
第二枚烟幕弹引爆时间B_1^2	0.01 s
第三枚烟幕弹的投放时间A_1^3	8.684366036886889 s
第三枚烟幕弹引爆时间B_1^3	0.01 s
决策变量θ	0.1473153513681911 rad
无人机飞行速度v_1	109.28990867142102 m/s

此时有效遮蔽时间最大，其数值为 6.38 秒，有效屏蔽时间的分布状况如下：

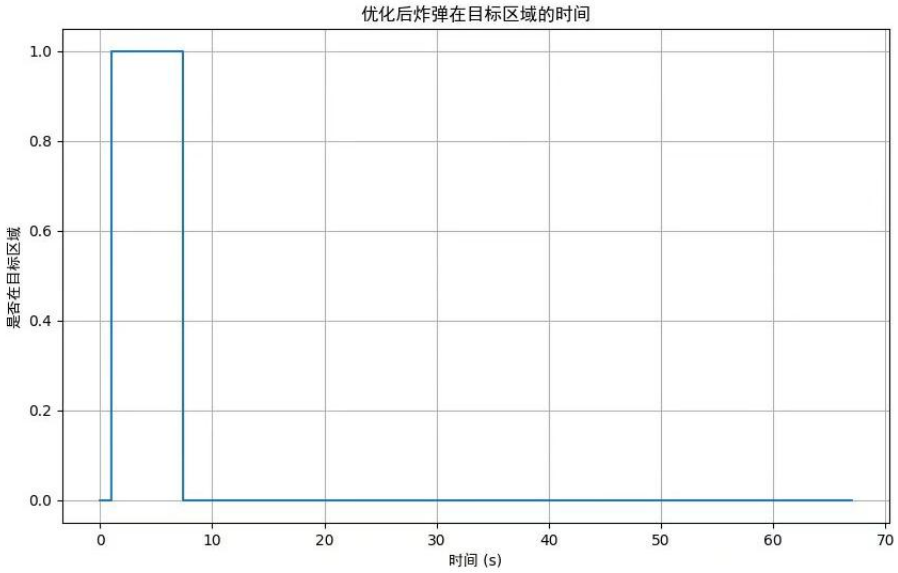

图 8 问题三有效屏蔽时间分布状况示意图

5.4 问题四模型的建立与求解

5.4.1 问题四模型的建立

问题四同样涉及多枚烟幕干扰弹对一个导弹的遮蔽有效时间计算，同问题三的方法采用一致的处理方式。且同样使用 PSO 算法进行求解。

（1）设定参数

在第四问中，有$FY1$、$FY2$、$FY3$三架无人机各自投放一枚烟幕干扰弹，则可设φ_j为第j辆无人机的航向角参数，v_j为第j辆无人机的速度，A_j^1为第j辆无人机的投放时间，B_j^1为第j架无人机的起爆时间。在t时刻，第j架无人机的烟幕干扰弹的云团中心位置为

$$\mathrm{n}_{\mathrm{j,t}}^{1}=\mathrm{FY}_{\mathrm{j}}+\left(\mathrm{A}_{\mathrm{j}}^{1}+\mathrm{B}_{\mathrm{j}}^{1}\right)*\varphi_{\mathrm{j}}*\mathrm{v}_{\mathrm{j}}-\frac{1}{2}*\mathrm{g}^{2}*(\mathrm{B}_{\mathrm{j}}^{1})^{2}*\mathrm{Z}-3*\mathrm{T}_{\mathrm{j}}^{1}*\mathrm{Y} \tag{32}$$

其中：

$$\mathrm{t}_{\mathrm{j}}=\mathrm{A}_{\mathrm{j}}^{1}+\mathrm{B}_{\mathrm{j}}^{1}+\mathrm{T}_{\mathrm{j}}^{1} \tag{33}$$

（2）目标函数

在多架无人机各自投放一枚烟幕弹的情况下，重新设置目标函数为：

$$\mathrm{Max}\ \sum_{\mathrm{j}=1}^{3}\mathrm{sumT}_{\mathrm{j}}^{1}=\sum_{\mathrm{j}=1}^{3}\int_{0}^{20}\mathrm{f}\left(\mathrm{T}_{\mathrm{j}}^{1}\right)\mathrm{dT}_{\mathrm{j}}^{1} \tag{34}$$

其中，$f(t)$为检验此时真目标是否被遮蔽的函数，即

$$\mathrm{f}\left(\mathrm{T}_{\mathrm{j}}^{1}\right)=\begin{cases}1, & \left(\mathrm{minL}_{\mathrm{j,t}}^{1,\mathrm{l}}\right)^{2}\leq\mathrm{R}^{2}\\ 0, & \left(\mathrm{minL}_{\mathrm{j,t}}^{1,\mathrm{l}}\right)^{2}>\mathrm{R}^{2}\end{cases},\ \mathrm{j}=1,2,3 \tag{35}$$

$$\mathrm{minL}_{\mathrm{j,t}}^{1,\mathrm{l}}=||\mathrm{key}_{1,\mathrm{t}}^{1,\mathrm{l}}+\mathrm{s}_{\mathrm{j,t}}^{1,\mathrm{l}'}*\left(\mathrm{M}_{1,\mathrm{t}}-\mathrm{key}_{1,\mathrm{t}}^{1,\mathrm{l}}\right)-\mathrm{n}_{\mathrm{j,t}}^{1}|| \tag{36}$$

$$\mathrm{s}_{\mathrm{j,t}}^{1,\mathrm{l}'}=\min\{1,\max\{0,\mathrm{s}_{\mathrm{j,t}}^{1,\mathrm{l}}\}\} \tag{37}$$

$$\mathrm{s}_{\mathrm{j,t}}^{1,\mathrm{l}}=\frac{(\mathrm{n}_{\mathrm{j,t}}^{1}-\mathrm{key}_{1,\mathrm{t}}^{1,\mathrm{l}})\cdot(\mathrm{M}_{1,\mathrm{t}}-\mathrm{key}_{1,\mathrm{t}}^{1,\mathrm{l}})}{\left|\left|\mathrm{M}_{1,\mathrm{t}}-\mathrm{key}_{1,\mathrm{t}}^{1,\mathrm{l}}\right|\right|^{2}} \tag{38}$$

（3）约束条件

根据题目要求，模型中隐含如下约束条件：

①保证投弹与引爆在导弹抵达假目标之前：

$$t\leq\frac{|F-M1|}{300}$$

②保证无人机在受领任务之后再投弹：

$$A_j^1\geq 0,\ j=1,2,3$$

③保证烟幕干扰弹在投弹之后再起爆：

$$B_j^1\geq 0,\ j=1,2,3$$

④云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内可为目标提供有效遮蔽：

$$T_j^1\in[0,20),\ j=1,2,3$$

⑤无人机飞行速度介于 70-140m/s 之间：

$$v_j \in [70，140],\ j = 1,2,3$$

（4）模型总括

$$\text{Max} \sum_{k=1}^{3} sumT_j^1 = \sum_{j=1}^{3} \int_0^{20} f(T_j^1) dT_j^1$$

$$s.t. \begin{cases} 0 < t \leq \dfrac{F - M1}{300} \\ v_j \in [70,140] \\ t_j = A_j^1 + B_j^1 + T_j^1 \\ T_j^1 \in [0,20) \\ 0 \leq A_j^1, B_j^1 \leq t \\ \theta_j \in [0，2\pi] \end{cases}$$

5.4.2 问题四模型的求解

使用 PSO 算法求解后，得到的最佳参数如下：

表 4：问题四变量最佳参数值

变量	最佳参数值	变量	最佳参数值
φ_1	0.08114642046476062	A_2^1	3.77826758523537
v_1	110.94255313860987	B_2^1	6.969946135493108
A_1^1	1.0745010369635817	φ_3	6.073019831924443
B_1^1	0.011240782584552194	v_3	140.0
φ_2	4.387090550188464	A_3^1	4.556191227927292
v_2	131.81971514614634	B_3^1	2.6471714833347337

此时有效遮蔽时间最大，其数值为 8.45 秒，有效屏蔽时间的分布状况如下：

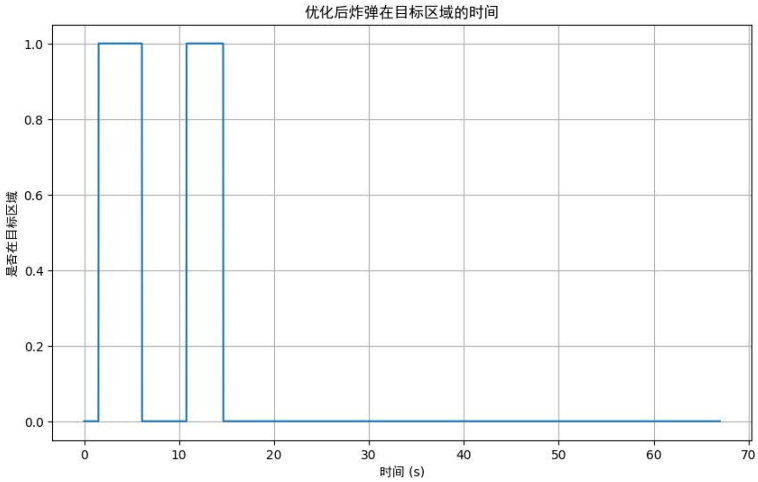

图 9 问题四有效屏蔽时间分布状况状态示意图

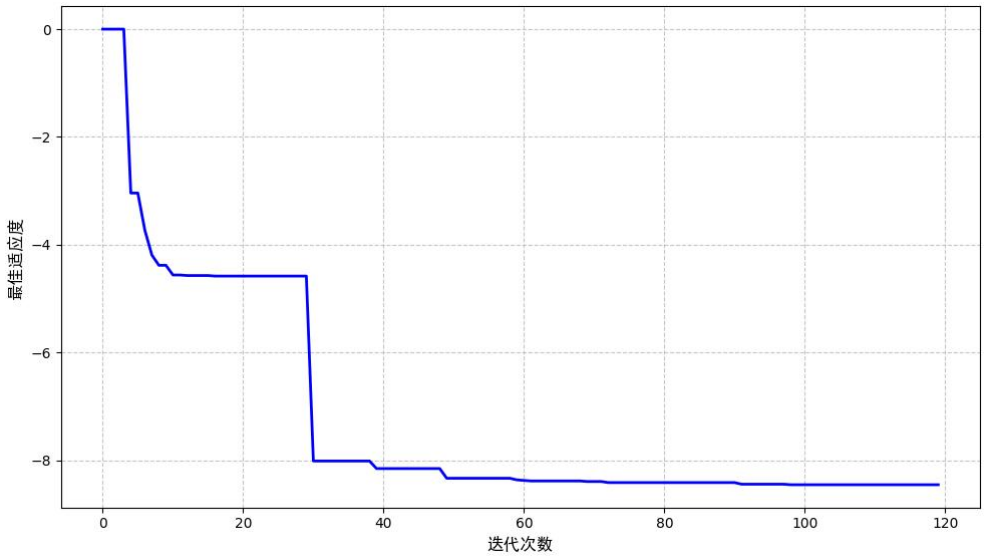

图 10　PSO 算法收敛曲线

5.5 问题五模型的建立与求解

5.5.1 问题五模型的建立

问题五涉及多个烟幕干扰弹对多个导弹视野的遮蔽问题，为避免重复计算单个导弹的有效遮蔽时间，同样建立统一的时间序列，把单个导弹在不同时刻的有效遮蔽时间整合到该序列上，此时序列取值为$\{0,1\}$。

在有多枚导弹时，我们定义有效遮蔽时间为有一枚或一枚以上导弹的视野被遮蔽的时间，且认为若某一时刻视野被遮蔽的导弹的数量越多则越有效，由此，我们可以将多个导弹的遮蔽时间序列相加，作为整体的有效遮蔽时间序列，其取值为有效遮蔽的程度，以此作为投放方案的度量。

（1）设定参数

在第五问中，有 5 架无人机至多投放 3 枚烟幕干扰弹，则设定第j辆无人机投放第k枚导弹的投放时间为A_j^k，第j辆无人机投放第k枚导弹的起爆时间为B_j^k，且设φ_j为第j辆无人机的航向角参数，v_j为第j架无人机的速度。在t时刻，第j架无人机的第k枚烟幕干扰弹的云团中心位置为：

$$n_{j,t}^k = FY_j + \left(A_j^k + B_j^k\right) * \varphi_j * v_j - \frac{1}{2} * g^2 * (B_j^k)^2 * Z - 3 * T_j^k * Y \tag{39}$$

其中：

$$t_j^k = A_j^k + B_j^k + T_j^k \tag{40}$$

（2）目标函数

在多架无人机各自投放多枚烟幕弹的情况下，重新设置目标函数为：

$$\text{Max} \sum_{j=1}^{5} \sum_{k=1}^{3} \text{sum}T_j^k = \sum_{j=1}^{5} \sum_{k=1}^{3} \int_0^{20} f\left(T_j^k\right) dT_j^k \tag{41}$$

其中，$f(t)$为检验此时真目标是否被遮蔽的函数，即

$$\mathrm{f}\left(\mathrm{T}_{\mathrm{j}}^{\mathrm{k}}\right)=\begin{cases}1, & \left(\mathrm{minL}_{\mathrm{j,t}}^{\mathrm{k,l}}\right)^2 \leq \mathrm{R}^2 \\ 0, & \left(\mathrm{minL}_{\mathrm{j,t}}^{\mathrm{k,l}}\right)^2 > \mathrm{R}^2\end{cases},\ \mathrm{j}=1,2,3,4,5,\ \mathrm{k}=1,2,3 \tag{42}$$

$$\mathrm{minL}_{\mathrm{j,t}}^{\mathrm{k,l}}=||\mathrm{key}_{\mathrm{i,t}}^{\mathrm{k,l}}+\mathrm{s}_{\mathrm{j,t}}^{\mathrm{k,l'}}*\left(\mathrm{M}_{\mathrm{i,t}}-\mathrm{key}_{\mathrm{i,t}}^{\mathrm{k,l}}\right)-\mathrm{n}_{\mathrm{j,t}}^{\mathrm{k}}|| \tag{43}$$

$$\mathrm{s}_{\mathrm{j,t}}^{\mathrm{k,l'}}=\min\{1,\max\{0,\mathrm{s}_{\mathrm{j,t}}^{\mathrm{k,l}}\}\} \tag{44}$$

$$\mathrm{s}_{\mathrm{j,t}}^{\mathrm{1,l}}=\frac{(\mathrm{n}_{\mathrm{j,t}}^{\mathrm{k}}-\mathrm{key}_{\mathrm{i,t}}^{\mathrm{k,l}})\cdot(\mathrm{M}_{\mathrm{i,t}}-\mathrm{key}_{\mathrm{i,t}}^{\mathrm{k,l}})}{\left|\left|\mathrm{M}_{\mathrm{i,t}}-\mathrm{key}_{\mathrm{i,t}}^{\mathrm{k,l}}\right|\right|^2} \tag{45}$$

（3）约束条件

根据题目要求，模型中隐含如下约束条件：

①保证投弹与引爆在导弹抵达假目标之前：

$$t \leq \frac{|F-M1|}{300}$$

②保证无人机在受领任务之后再投弹：

$$A_j^k \geq 0,\ j=1,2,3,4,5,\ k=1,2,3$$

③保证烟幕干扰弹在投弹之后再起爆：

$$B_j^k \geq 0,\ j=1,2,3,4,5,\ k=1,2,3$$

④云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内可为目标提供有效遮蔽：

$$T_j^k \in [0,20),\ j=1,2,3,4,5,\ k=1,2,3$$

⑤无人机飞行速度介于 70-140m/s 之间：

$$v_j \in [70,\ 140],\ j=1,2,3,4,$$

⑥第一枚、第二枚、第三枚烟幕干扰弹的投放有先后顺序，且两枚导弹投放间隔超过 1s：

$$A_j^1+1<A_j^2,\ j=1,2,3,4,5$$

$$A_j^2+1<A_j^3,\ j=1,2,3,4,5$$

（4）模型总括

$$\mathrm{Max}\sum_{j=1}^{5}\sum_{k=1}^{3}\mathrm{sumT}_j^k=\sum_{j=1}^{5}\sum_{k=1}^{3}\int_0^{20}f\left(T_j^k\right)dT_j^k$$

$$
s.t.\begin{cases} 0 < t_j^k \le \dfrac{F-M1}{300} \\ v_j \in [70,140] \\ t_j^k = A_j^k + B_j^k + T_j^k \\ T_j^k \in [0,20) \\ 0 \le A_j^k, B_j^k \le t \\ \theta_j \in [0,\ 2\pi] \\ A_j^1 + 1 < A_j^2 \\ A_j^2 + 1 < A_j^3 \end{cases}
$$

5.5.2 问题五模型的求解

采用 PSO 算法对模型求解后，得到的最佳参数结果如下，此时全局遮蔽时长为 33.01s。

表 5：问题五变量最佳参数值

变量	最佳参数值	变量	最佳参数值
A_1^1	0.01	B_1^2	0.01
A_2^1	7	B_2^2	0.01
A_3^1	7	B_3^2	1.99878705
A_4^1	2.05737073	B_4^2	5.03075984
A_5^1	0.163988328	B_5^2	0.01
A_1^2	1.32653047	B_1^3	4.65137563
A_2^2	4.97578143	B_2^3	4.62931181
A_3^2	1	B_3^3	0.025549731
A_4^2	7	B_4^3	7
A_5^2	6.41023581	B_5^3	1.49887622
A_1^3	6.89512977	φ_1	6.28318531
A_2^3	3.08911589	φ_2	5.26630367
A_3^3	2.15899613	φ_3	6.27123568
A_4^3	2	φ_4	3.45195085
A_5^3	7	φ_5	4.10963751
B_1^1	0.01	v_1	97.9397308
B_2^1	7	v_2	95.39954
B_3^1	0.01526927	v_3	70
B_4^1	6.99971738	v_4	139.909788
B_5^1	6.9859429	v_5	121.894455

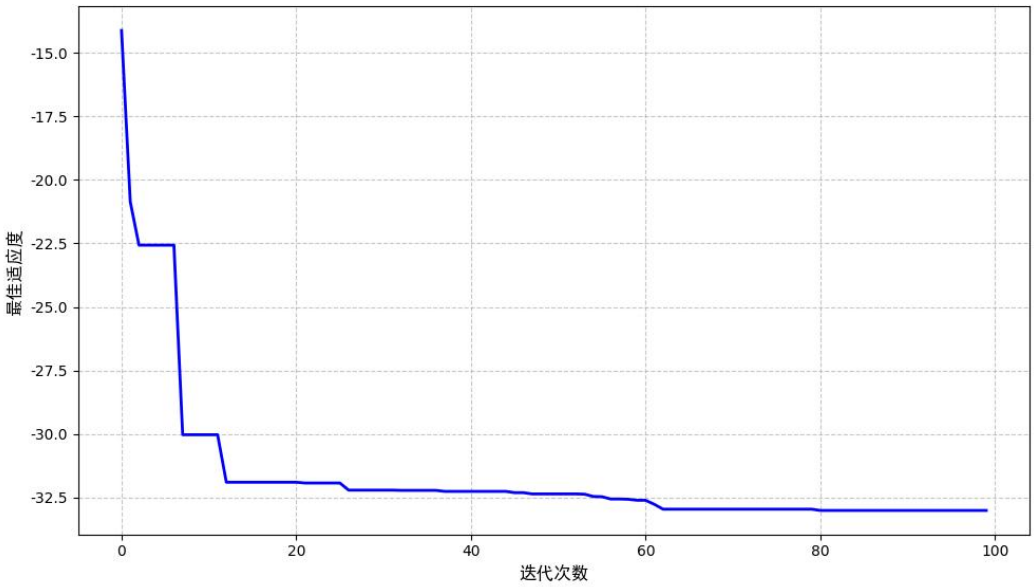

图 11 PSO 算法收敛曲线

六、 模型的优缺点与改进

6.1 模型的优点

① 通用性强，适用范围广。混合整数规划模型能够同时处理整数变量和连续变量，适合描述具有离散决策和连续过程相结合的复杂优化问题，具有广泛的适用性。

② 数学表达清晰、严谨。混合整数规划模型结构严谨，可用数学公式准确表达约束条件和目标函数，便于进行理论分析、模型验证，同时有利于扩展和改进后续求解方法。

6.2 模型的缺点

① 计算复杂度高，随着问题规模增大，混合整数规划模型求解时间呈指数级增长，尤其是在变量和约束数量较多时，计算效率低下，难以满足实时性要求。

② 对问题数据敏感，混合整数规划模型的可行性和求解结果对输入数据的准确性高度敏感，若参数存在误差或不确定性，可能导致模型失效或求解效率显著降低。

6.3 模型的改进

① 引入分解策略，对于大规模问题，可将原问题分解为多个子问题，采用分块求解，降低计算复杂度，提高求解效率。

② 结合启发式方法，将混合整数规划模型与启发式或元启发式算法结合，例如使用模拟退火或遗传算法生成初始解，再利用数学规划进行精确求解，从而加快收敛。

③ 考虑不确定性优化，针对参数不确定问题，引入鲁棒优化或随机规划，提高模型对数据误差的容忍度和实用性。

七、 参考文献

[1]陈浩,高欣宝,李天鹏,等. 国外烟幕干扰弹发展及关键技术研究 [J]. 飞航导弹, 2017, (12): 71-74. 2017.12.16.

[2]吴鸿超,梁增友,邓德志,等. 烟幕干扰弹新型战斗部结构研究 [J]. 科学技术与工程, 2016, 16 (01): 183-187.

[3]范鑫磊,李栋,邹杰. 基于 GA-BP 算法的直升机红外诱饵弹投放策略训练研究 [J/OL]. 电光与控制, 1-6[2025-09-05].

[4]陈书恒,朱倪瑶,陈宁,等. 飞机红外辐射特性测试及红外干扰弹投放方式研究 [J]. 红外技术, 2021, 43 (10): 949-953.

[5]王宇,杨光. 水面舰艇抗无源干扰方案优化设计与实现 [J]. 计算机与数字工程, 2017, 45 (12): 2542-2546.

[6]翟梦月,王征,何家宁,等. 移动货架仓库基于巷道均衡的货架选择优化[J/OL].工程管理科技前沿,1-12[2025-09-06].

[7]唐亮,王清阁,谭真,等. 不确定交货期下供应链协同鲁棒调度优化[J].管理科学学报,2025,28(06):103-118.

[8]镇璐,高佳静,谭哲一. 多卡车与多无人机协同配送路径优化问题研究[J].管理科学学报,2024,27(07):1-18.

[9]周忠宝,陈恩铭,李瑞阳,等. 基于多无人机的血液紧急配送问题研究[J/OL].中国管理科学,1-12[2025-09-06].

[10]杨吉威,王征,王艺雪,等. 移动货架仓库中搬运机器人优化调度模型与算法[J].工程管理科技前沿,2023,42(04):18-26.

[11]蔡沛沛,蔡耿生,林业伟,等. 热处理对石墨烯增强 Ti-6Al-4V 合金显微组织和硬度的影响[J].热加工工艺,2025,54(09):27-32.

[12]汤育玺,王进,崔明磊,等. 双重退火工艺对准β锻 TC18 合金锻件组织和性能的影响[J].金属热处理,2024,49(11):262-266.

[13]肖挺,马斌,宣航. 一次退火冷却方式对 TC25 钛合金锻件组织与性能的影响[J].金属热处理,2024,49(10):118-120.

[14]高潘,曾卫东,张传臣,等. 双重退火对 TC4-DT/TC21 线性摩擦焊接头组织及力学性能影响[J].稀有金属,2024,48(01):1-10.

附录

附录 1
问题一求解

```
import numpy as np
from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from scipy.optimize import dual_annealing
plt.rcParams["font.family"] = ["SimHei"]
plt.rcParams["axes.unicode_minus"] = False

def direction(A,B):
    AB=np.array(B)-np.array(A)
    return AB/np.linalg.norm(AB)

def test1(M_position,n_jkt):
    direction_vec=direction(M_position,True_obj)
    #投影
    p_vec=np.array([direction_vec[0],direction_vec[1],0])
    p_vec=p_vec/np.linalg.norm(p_vec)
    p_vec_T=np.array([p_vec[1],-p_vec[0],0])
    #关键点
    key1=True_obj+p_vec*7+10*np.array([0,0,1])
    key2=True_obj-p_vec*7+10*np.array([0,0,1])
    key3=True_obj+p_vec_T*7+10*np.array([0,0,1])
    key4=True_obj-p_vec_T*7+10*np.array([0,0,1])
    key5=key3-5*np.array([0,0,1])
    key6=key4-5*np.array([0,0,1])
    key7=True_obj-p_vec*7
    key8=True_obj+p_vec_T*7
    key9=True_obj-p_vec_T*7
    keys=[key1,key2,key3,key4,key5,key6,key7,key8,key9]
    #检验
    for i in keys:
        s=np.dot((n_jkt-i),(M_position-i))/np.linalg.norm(M_position-i)**2
```

```
        s_=min(1,max(0,s))
        d2=np.linalg.norm(i+s_*(M_position-i)-n_jkt)**2
        if d2<=100:
            continue
        else:
            return 0
    return 1

True_obj=[0,200,0]
F=[0,0,0]
M1=[20000,0,2000]
M2=[19000,600,2100]
M3=[18000,-600,700]
FY1=[17800,0,1800]
FY2=[12000,1400,1400]
FY3=[6000,-3000,700]
FY4=[11000,2000,1800]
FY5=[13000,-2000,1300]
a=1.5
b=3.6
v_0=300
v_fy1=120

#时间段
A=np.arange(0,a,0.001)
B=np.arange(0,b,0.001)
T=np.arange(0,20,0.001)
t=a+b+T

D1=direction(M1,F) #导弹方向向量
D2=direction([17800,0,0],[0,0,0]) #无人机方向向量

n_jkt=[]
M1_t=[]
result=[]
```

```
N=[]
for i in t:
    n_jkt=np.array(FY1)+(a+b)*v_fy1*D2-0.5*9.8*(b)**2*np.array([0,0,1])-3*(i-a-b)*np.array([0,0,1])
    M1_t=np.array(M1)+D1*v_0*i
    result.append(test1(M1_t,n_jkt))
    N.append(n_jkt)  # 记录每个时间点的轰炸机位置

result=np.array(result)
fig=plt.figure(figsize=(10, 6))
plt.plot(T,result)
plt.xlabel('时间 (s)')
plt.grid(True)
plt.ylabel('是否在目标区域')

print('有限遮蔽时间为：')
print(sum(result*(20/20000)))
```

问题二求解

```
True_obj = np.array([0, 200, 0])
F = np.array([0, 0, 0])
M1 = np.array([20000, 0, 2000])
M2 = np.array([19000, 600, 2100])
M3 = np.array([18000, -600, 700])
FY1 = np.array([17800, 0, 1800])

def objective_function(x):
    # 解包参数: 无人机速度, 投放时间, 起爆时间, 无人机航向角
    v_drone, A, B, theta = x

    # 无人机飞行方向 (水平于 xy 平面)
    drone_direction = np.array([np.cos(theta), np.sin(theta), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction = drone_direction
```

```
    # 导弹飞行方向
    missile_direction = direction(M1, F)
    v_missile = 300   # 导弹速度 300m/s

    # 时间参数
    T = np.linspace(0, 20, 2000)
    t = A + B + T

    # 计算结果
    result = []
    for i in t:
        # 计算烟幕弹位置 (基于原始代码的计算方式)
        n_jkt = np.array(FY1) + (A + B) * v_drone * bomb_direction - 0.5 * 9.8 * (B) ** 2
* np.array([0, 0, 1]) - 3 * (
                        i - A - B) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction * v_missile * i

        # 检验
        result.append(test1(M1_t, n_jkt))

    result = np.array(result)
    total_time = np.sum(result) * (20 / 2000)

    # 我们希望最大化总时间，但优化算法默认最小化，所以取负
    return -total_time


# 定义参数边界
# v_drone, A, B, theta
bounds = [(70, 140), (0.1, 5), (0.1, 5), (0, 2 * np.pi)]

# 使用双模拟退火算法进行优化
```

```
result = dual_annealing(objective_function, bounds, maxiter=100, seed=42)
xopt = result.x
fopt = result.fun

print("模拟退火优化结果:")
print(f"最佳参数: 无人机速度={xopt[0]} m/s, 投放时间={xopt[1]} s, 起爆时间
={xopt[2]} s, 航向角={xopt[3]} rad")
print(f"最大有效遮蔽时间: {-fopt} 秒")

# 使用最佳参数计算并可视化结果
v_drone_opt, A_opt, B_opt, theta_opt = xopt
drone_direction_opt = np.array([np.cos(theta_opt), np.sin(theta_opt), 0])
bomb_direction_opt = drone_direction_opt   # 投弹方向与无人机方向一致

# 导弹飞行方向
missile_direction = direction(M1, F)
v_missile = 300

# 时间参数
T = np.linspace(0, 20, 2000)
t = A_opt + B_opt + T
result_opt = []

for i in t:
    n_jkt = np.array(FY1) + (A_opt + B_opt) * v_drone_opt * bomb_direction_opt - 0.5 *
9.8 * (B_opt) ** 2 * np.array(
        [0, 0, 1]) - 3 * (i - A_opt - B_opt) * np.array([0, 0, 1])
    M1_t = np.array(M1) + missile_direction * v_missile * i
    result_opt.append(test1(M1_t, n_jkt))

result_opt = np.array(result_opt)

plt.figure(figsize=(10, 6))
plt.plot(T, result_opt)
plt.xlabel('时间 (s)')
```

```
plt.ylabel('是否在目标区域')
plt.title('优化后炸弹在目标区域的时间')
plt.grid(True)
plt.show()

print(f"优化后在目标区域总时间: {np.sum(result_opt) * (20 / 2000)}  秒")
```

问题三求解

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# 移除可能导致问题的字体设置
plt.rcParams["font.family"] = ["DejaVu Sans", "Arial Unicode MS", "SimHei"]

# 初始坐标
True_obj = np.array([0, 200, 0])
F = np.array([0, 0, 0])
M1 = np.array([20000, 0, 2000])
M2 = np.array([19000, 600, 2100])
M3 = np.array([18000, -600, 700])
FY1 = np.array([17800, 0, 1800])


# 计算方向向量
def direction(A, B):
    AB = np.array(B) - np.array(A)
    return AB / np.linalg.norm(AB)


# 检验函数
def test1(M_position, n_jkt):
    direction_vec = direction(M_position, True_obj)
    # 投影
    p_vec = np.array([direction_vec[0], direction_vec[1], 0])
```

```
    p_vec = p_vec / np.linalg.norm(p_vec)
    p_vec_T = np.array([p_vec[1], -p_vec[0], 0])
    # 关键点
    key1 = True_obj + p_vec * 7 + 10 * np.array([0, 0, 1])
    key2 = True_obj - p_vec * 7 + 10 * np.array([0, 0, 1])
    key3 = True_obj + p_vec_T * 7 + 10 * np.array([0, 0, 1])
    key4 = True_obj - p_vec_T * 7 + 10 * np.array([0, 0, 1])
    key5 = key3 - 5 * np.array([0, 0, 1])
    key6 = key4 - 5 * np.array([0, 0, 1])
    key7 = True_obj - p_vec * 7
    key8 = True_obj + p_vec_T * 7
    key9 = True_obj - p_vec_T * 7
    keys = [key1, key2, key3, key4, key5, key6, key7, key8, key9]
    # 检验
    for i in keys:
        s = np.dot((n_jkt - i), (M_position - i)) / np.linalg.norm(M_position - i) ** 2
        s_ = min(1, max(0, s))
        d2 = np.linalg.norm(i + s_ * (M_position - i) - n_jkt) ** 2
        if d2 <= 100:
            continue
        else:
            return 0
    return 1


# 目标函数 - 计算有效遮蔽时间
def objective_function(x):
    # 解包参数: 无人机速度, 无人机航向角,第一枚投放时间, 第一枚起爆时间, 投放时间 2，引爆时间 2，投放时间 3，引爆时间 3
    v_drone, theta, A1, B1, A2, B2, A3, B3 = x

    # 无人机飞行方向 (水平于 xy 平面)
    drone_direction = np.array([np.cos(theta), np.sin(theta), 0])

    # 投弹方向与无人机飞行方向一致
```

```
    bomb_direction = drone_direction

    # 导弹飞行方向
    missile_direction = direction(M1, F)
    v_missile = 300   # 导弹速度 300m/s

    # 时间参数
    total_missile_time = np.linalg.norm(M1 - F) / v_missile
    T = np.arange(0, total_missile_time, 0.01)

    # 计算结果 - 直接使用列表，不使用 pandas Series
    timeline = np.zeros(len(T))

    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A1 + B1:
            continue

        # 计算烟幕弹位置
        n_jkt1 = np.array(FY1) + (A1 + B1) * v_drone * bomb_direction - 0.5 * 9.8 * (B1)
** 2 * np.array(
            [0, 0, 1]) - 3 * (
                                t - A1 - B1) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction * v_missile * t

        # 检验
        if test1(M1_t, n_jkt1):
            timeline[i] += 1
        count1 += 1
```

```
    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A1 + A2 + B2:
            continue

        # 计算烟幕弹位置
        n_jkt2 = np.array(FY1) + (A1 + A2 + B2) * v_drone * bomb_direction - 0.5 * 9.8 *
(B2) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A1 - B2 - A2) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction * v_missile * t

        # 检验
        if test1(M1_t, n_jkt2):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A1 + A2 + A3 + B3:
            continue

        # 计算烟幕弹位置
        n_jkt3 = np.array(FY1) + (A1 + A2 + A3 + B3) * v_drone * bomb_direction - 0.5 *
9.8 * (B3) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A1 - B3 - A2 - A3) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction * v_missile * t
```

```
            # 检验
            if test1(M1_t, n_jkt3):
                timeline[i] += 1
            count3 += 1

        # 确保值不超过 1
        timeline = np.clip(timeline, 0, 1)

        total_time = np.sum(timeline) * 0.01    # 每个时间点代表 0.01 秒

        # 我们希望最大化总时间，但优化算法默认最小化，所以取负
        return -total_time


# 定义参数边界
# v_drone, theta, A1, B1, A2, B2, A3, B3
bounds = [(70, 140), (0, 2 * np.pi), (0.01, 14), (0.01, 14), (1, 14), (0.01, 14), (2, 14), (0.01, 14)]


# PSO 算法实现
class PSO:
    def __init__(self, objective_func, bounds, num_particles=40, max_iter=150, w=0.7, c1=1.5, c2=1.5):
        self.objective_func = objective_func
        self.bounds = bounds
        self.num_particles = num_particles
        self.max_iter = max_iter
        self.w = w    # 惯性权重
        self.c1 = c1    # 个体学习因子
        self.c2 = c2    # 社会学习因子
        self.dim = len(bounds)

        # 初始化粒子位置和速度
        self.X = np.random.uniform(low=[b[0] for b in bounds],
```

```
                                        high=[b[1] for b in bounds],
                                        size=(num_particles, self.dim))
        self.V = np.random.uniform(-1, 1, (num_particles, self.dim))

        # 初始化个体最优位置和适应度
        self.P = self.X.copy()
        self.p_fitness = np.array([self.objective_func(x) for x in self.X])

        # 初始化全局最优位置和适应度
        self.g = self.P[np.argmin(self.p_fitness)]
        self.g_fitness = np.min(self.p_fitness)

        # 记录每次迭代的最佳适应度
        self.fitness_history = []

    def optimize(self):
        for iter in range(self.max_iter):
            # 更新惯性权重（线性递减）
            w = self.w * (1 - iter / self.max_iter)

            for i in range(self.num_particles):
                # 更新速度
                r1, r2 = np.random.rand(self.dim), np.random.rand(self.dim)
                self.V[i] = (w * self.V[i] +
                             self.c1 * r1 * (self.P[i] - self.X[i]) +
                             self.c2 * r2 * (self.g - self.X[i]))

                # 更新位置
                self.X[i] = self.X[i] + self.V[i]

                # 边界处理
                for j in range(self.dim):
                    if self.X[i, j] < self.bounds[j][0]:
                        self.X[i, j] = self.bounds[j][0]
                    elif self.X[i, j] > self.bounds[j][1]:
```

```
                self.X[i, j] = self.bounds[j][1]

            # 计算新位置的适应度
            fitness = self.objective_func(self.X[i])

            # 更新个体最优
            if fitness < self.p_fitness[i]:
                self.P[i] = self.X[i].copy()
                self.p_fitness[i] = fitness

            # 更新全局最优
            if fitness < self.g_fitness:
                self.g = self.X[i].copy()
                self.g_fitness = fitness

        # 记录当前迭代的最佳适应度
        self.fitness_history.append(self.g_fitness)

        # 打印当前迭代信息
        if (iter + 1) % 10 == 0:
            print(f"Iteration {iter + 1}/{self.max_iter}, Best Fitness: {-self.g_fitness:.4f}")

    return self.g, self.g_fitness


# 使用 PSO 算法进行优化
print("开始 PSO 优化...")
pso = PSO(objective_function, bounds, num_particles=30, max_iter=100)
xopt, fopt = pso.optimize()

print("PSO 优化结果:")
print(
    f"最佳参数: 无人机速度={xopt[0]} m/s, 投放时间 1={xopt[2]} s, 起爆时间 1={xopt[3]} s, 航向角={xopt[1]} rad, 投放时间 2={xopt[4]} s, 起爆时间 2={xopt[5]} s,
```

```
投放时间 3={xopt[6]} s, 起爆时间 3={xopt[7]} s")
print(f"最大有效遮蔽时间: {-fopt} 秒")

# 绘制适应度变化曲线
plt.figure(figsize=(10, 6))
plt.plot(range(1, len(pso.fitness_history) + 1), [-f for f in pso.fitness_history])
plt.xlabel('迭代次数')
plt.ylabel('最佳适应度')
plt.title('PSO 优化过程')
plt.grid(True)
plt.show()

# 使用最佳参数计算并可视化结果
v_drone_opt, theta_opt, A1_opt, B1_opt, A2_opt, B2_opt, A3_opt, B3_opt = xopt
drone_direction_opt = np.array([np.cos(theta_opt), np.sin(theta_opt), 0])
bomb_direction_opt = drone_direction_opt   # 投弹方向与无人机方向一致

# 导弹飞行方向
missile_direction = direction(M1, F)
v_missile = 300
total_missile_time = np.linalg.norm(M1 - F) / v_missile

# 时间参数
T = np.arange(0, total_missile_time, 0.01)
timeline = np.zeros(len(T))

count1, count2, count3 = 0, 0, 0
# 第一个烟幕弹
for i, t in enumerate(T):
    if count1 > 2000:
        break
    if t < A1_opt + B1_opt:
        continue

    # 计算烟幕弹位置
```

```
    n_jkt1 = np.array(FY1) + (A1_opt + B1_opt) * v_drone_opt * bomb_direction_opt - 0.5
* 9.8 * (
        B1_opt) ** 2 * np.array([0, 0, 1]) - 3 * (
                        t - A1_opt - B1_opt) * np.array([0, 0, 1])

    # 计算导弹位置
    M1_t = np.array(M1) + missile_direction * v_missile * t

    # 检验
    if test1(M1_t, n_jkt1):
        timeline[i] += 1
    count1 += 1

# 第二个烟幕弹
for i, t in enumerate(T):
    if count2 > 2000:
        break
    if t < A1_opt + A2_opt + B2_opt:
        continue

    # 计算烟幕弹位置
    n_jkt2 = np.array(FY1) + (A1_opt + A2_opt + B2_opt) * v_drone_opt *
bomb_direction_opt - 0.5 * 9.8 * (
        B2_opt) ** 2 * np.array(
        [0, 0, 1]) - 3 * (t - A1_opt - B2_opt - A2_opt) * np.array([0, 0, 1])

    # 计算导弹位置
    M1_t = np.array(M1) + missile_direction * v_missile * t

    # 检验
    if test1(M1_t, n_jkt2):
        timeline[i] += 1
    count2 += 1

# 第三个烟幕弹
```

```
for i, t in enumerate(T):
    if count3 > 2000:
        break
    if t < A1_opt + A2_opt + A3_opt + B3_opt:
        continue

    # 计算烟幕弹位置
    n_jkt3 = np.array(FY1) + (A1_opt + A2_opt + A3_opt + B3_opt) * v_drone_opt * bomb_direction_opt - 0.5 * 9.8 * (
        B3_opt) ** 2 * np.array(
        [0, 0, 1]) - 3 * (t - A1_opt - B3_opt - A2_opt - A3_opt) * np.array([0, 0, 1])

    # 计算导弹位置
    M1_t = np.array(M1) + missile_direction * v_missile * t

    # 检验
    if test1(M1_t, n_jkt3):
        timeline[i] += 1
    count3 += 1

# 确保值不超过 1
timeline = np.clip(timeline, 0, 1)

total_time = np.sum(timeline) * 0.01

plt.figure(figsize=(10, 6))
plt.plot(T, timeline)
plt.xlabel('时间 (s)')
plt.ylabel('是否在目标区域')
plt.title('优化后炸弹在目标区域的时间')
plt.grid(True)
plt.show()

print(f"优化后在目标区域总时间: {total_time} 秒")
```

问题四求解

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import dual_annealing

# 移除可能导致问题的字体设置
plt.rcParams["font.family"] = ["DejaVu Sans", "Arial Unicode MS", "SimHei"]

# 初始坐标
True_obj = np.array([0, 200, 0])
F = np.array([0, 0, 0])
M1 = np.array([20000, 0, 2000])
M2 = np.array([19000, 600, 2100])
M3 = np.array([18000, -600, 700])
FY1 = np.array([17800, 0, 1800])
FY2 = np.array([12000, 1400, 1400])
FY3 = np.array([6000, -3000, 700])

# 计算方向向量
def direction(A, B):
    AB = np.array(B) - np.array(A)
    return AB / np.linalg.norm(AB)


# 检验函数
def test1(M_position, n_jkt):
    direction_vec = direction(M_position, True_obj)
    # 投影
    p_vec = np.array([direction_vec[0], direction_vec[1], 0])
    p_vec = p_vec / np.linalg.norm(p_vec)
    p_vec_T = np.array([p_vec[1], -p_vec[0], 0])
    # 关键点
    key1 = True_obj + p_vec * 7 + 10 * np.array([0, 0, 1])
    key2 = True_obj - p_vec * 7 + 10 * np.array([0, 0, 1])
    key3 = True_obj + p_vec_T * 7 + 10 * np.array([0, 0, 1])
```

```
    key4 = True_obj - p_vec_T * 7 + 10 * np.array([0, 0, 1])
    key5 = key3 - 5 * np.array([0, 0, 1])
    key6 = key4 - 5 * np.array([0, 0, 1])
    key7 = True_obj - p_vec * 7
    key8 = True_obj + p_vec_T * 7
    key9 = True_obj - p_vec_T * 7
    keys = [key1, key2, key3, key4, key5, key6, key7, key8, key9]
    # 检验
    for i in keys:
        s = np.dot((n_jkt - i), (M_position - i)) / np.linalg.norm(M_position - i) ** 2
        s_ = min(1, max(0, s))
        d2 = np.linalg.norm(i + s_ * (M_position - i) - n_jkt) ** 2
        if d2 <= 100:
            continue
        else:
            return 0
    return 1


# 目标函数 - 计算有效遮蔽时间
def objective_function(x):
    # 解包参数: 无人机速度 1, 无人机航向角 1,投放时间 1,起爆时间 1, 无人机速度 2,
无人机航向角 2,投放时间 2,起爆时间 2,无人机速度 3, 无人机航向角 3,投放时间 3,起爆
时间 3
    v_drone1, theta1, A1, B1,v_drone2, theta2, A2, B2,v_drone3, theta3, A3, B3 = x

    # 导弹飞行方向
    missile_direction = direction(M1, F)
    v_missile = 300   # 导弹速度 300m/s

    # 无人机 1 飞行方向 (水平于 xy 平面)
    drone_direction1 = np.array([np.cos(theta1), np.sin(theta1), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction1 = drone_direction1
```

```
# 无人机 2 飞行方向 (水平于 xy 平面)
drone_direction2 = np.array([np.cos(theta2), np.sin(theta2), 0])

# 投弹方向与无人机飞行方向一致
bomb_direction2 = drone_direction2

# 无人机 3 飞行方向 (水平于 xy 平面)
drone_direction3 = np.array([np.cos(theta3), np.sin(theta3), 0])

# 投弹方向与无人机飞行方向一致
bomb_direction3 = drone_direction3


# 时间参数
total_missile_time = np.linalg.norm(M1 - F) / v_missile
T = np.arange(0, total_missile_time, 0.01)

# 计算结果 - 直接使用列表，不使用 pandas Series
timeline = np.zeros(len(T))

count1, count2, count3 = 0, 0, 0
# 第一个烟幕弹
for i, t in enumerate(T):
    if count1 > 2000:
        break
    if t < A1 + B1:
        continue

    # 计算烟幕弹位置
    n_jkt1 = np.array(FY1) + (A1 + B1) * v_drone1 * bomb_direction1 - 0.5 * 9.8 *
(B1) ** 2 * np.array(
        [0, 0, 1]) - 3 * (
                    t - A1 - B1) * np.array([0, 0, 1])
```

```
        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction * v_missile * t

        # 检验
        if test1(M1_t, n_jkt1):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A2 +B2:
            continue

        # 计算烟幕弹位置
        n_jkt2 = np.array(FY2) + (A2 + B2) * v_drone2 * bomb_direction2 - 0.5 * 9.8 *
(B2) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - B2 - A2) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction * v_missile * t

        # 检验
        if test1(M1_t, n_jkt2):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A3 + B3:
            continue
```

```
            # 计算烟幕弹位置
            n_jkt3 = np.array(FY3) + ( A3 + B3) * v_drone3 * bomb_direction3 - 0.5 * 9.8 *
(B3) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - B3 - A3) * np.array([0, 0, 1])

            # 计算导弹位置
            M1_t = np.array(M1) + missile_direction * v_missile * t

            # 检验
            if test1(M1_t, n_jkt3):
                timeline[i] += 1
            count3 += 1

        # 确保值不超过 1
        timeline = np.clip(timeline, 0, 1)

        total_time = np.sum(timeline) * 0.01    # 每个时间点代表 0.01 秒

        # 我们希望最大化总时间，但优化算法默认最小化，所以取负
        return -total_time


# 定义参数边界
# v_drone1, theta1, A1, B1,v_drone2, theta2, A2, B2,v_drone3, theta3, A3, B3
bounds = [(70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7), (70, 140), (0, 2 * np.pi), (0.01, 7),
(0.01, 7),(70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7)]



# PSO 算法实现
class PSO:
    def __init__(self, objective_func, bounds, num_particles=40, max_iter=260, w=0.9,
c1=1.5, c2=1.5):
        self.objective_func = objective_func
        self.bounds = bounds
```

```
        self.num_particles = num_particles
        self.max_iter = max_iter
        self.w = w    # 惯性权重
        self.c1 = c1    # 个体学习因子
        self.c2 = c2    # 社会学习因子
        self.dim = len(bounds)

        # 初始化粒子位置和速度
        self.X = np.random.uniform(low=[b[0] for b in bounds],
                                   high=[b[1] for b in bounds],
                                   size=(num_particles, self.dim))
        self.V = np.random.uniform(-1, 1, (num_particles, self.dim))

        # 初始化个体最优位置和适应度
        self.P = self.X.copy()
        self.p_fitness = np.array([self.objective_func(x) for x in self.X])

        # 初始化全局最优位置和适应度
        self.g = self.P[np.argmin(self.p_fitness)]
        self.g_fitness = np.min(self.p_fitness)

        # 记录每次迭代的最佳适应度
        self.fitness_history = []

    def optimize(self):
        for iter in range(self.max_iter):
            # 更新惯性权重（线性递减）
            w = self.w * (1 - iter / self.max_iter)

            for i in range(self.num_particles):
                # 更新速度
                r1, r2 = np.random.rand(self.dim), np.random.rand(self.dim)
                self.V[i] = (w * self.V[i] +
                             self.c1 * r1 * (self.P[i] - self.X[i]) +
                             self.c2 * r2 * (self.g - self.X[i]))
```

```
                # 更新位置
                self.X[i] = self.X[i] + self.V[i]

                # 边界处理
                for j in range(self.dim):
                    if self.X[i, j] < self.bounds[j][0]:
                        self.X[i, j] = self.bounds[j][0]
                    elif self.X[i, j] > self.bounds[j][1]:
                        self.X[i, j] = self.bounds[j][1]

                # 计算新位置的适应度
                fitness = self.objective_func(self.X[i])

                # 更新个体最优
                if fitness < self.p_fitness[i]:
                    self.P[i] = self.X[i].copy()
                    self.p_fitness[i] = fitness

                # 更新全局最优
                if fitness < self.g_fitness:
                    self.g = self.X[i].copy()
                    self.g_fitness = fitness

            # 记录当前迭代的最佳适应度
            self.fitness_history.append(self.g_fitness)

            # 打印当前迭代信息
            if (iter + 1) % 10 == 0:
                print(f"Iteration {iter + 1}/{self.max_iter}, Best Fitness: {-self.g_fitness:.4f}")

        return self.g, self.g_fitness
```

```
# 使用 PSO 算法进行优化
print("开始 PSO 优化...")
pso = PSO(objective_function, bounds, num_particles=50, max_iter=250)
xopt, fopt = pso.optimize()

print("PSO 优化结果:")
print(
    f"最佳参数: 无人机速度 1:{xopt[0]}, 无人机航向角 1:{xopt[1]},投放时间 1:{xopt[2]},起爆时间 1:{xopt[3]}, 无人机速度 2:{xopt[4]}, 无人机航向角 2:{xopt[5]},投放时间 2:{xopt[6]},起爆时间 2:{xopt[7]},无人机速度 3:{xopt[8]}, 无人机航向角 3:{xopt[9]},投放时间 3:{xopt[10]},起爆时间 3:{xopt[11]}")
print(f"最大有效遮蔽时间: {-fopt} 秒")

# 使用最佳参数计算并可视化结果
v_drone1_opt, theta1_opt, A1_opt, B1_opt, v_drone2_opt, theta2_opt, A2_opt, B2_opt, v_drone3_opt, theta3_opt, A3_opt, B3_opt = xopt

# 导弹飞行方向
missile_direction = direction(M1, F)
v_missile = 300   # 导弹速度 300m/s

# 无人机 1 飞行方向 (水平于 xy 平面)
drone_direction1_opt = np.array([np.cos(theta1_opt), np.sin(theta1_opt), 0])

# 投弹方向与无人机飞行方向一致
bomb_direction1_opt = drone_direction1_opt

# 无人机 2 飞行方向 (水平于 xy 平面)
drone_direction2_opt = np.array([np.cos(theta2_opt), np.sin(theta2_opt), 0])

# 投弹方向与无人机飞行方向一致
bomb_direction2_opt = drone_direction2_opt

# 无人机 3 飞行方向 (水平于 xy 平面)
drone_direction3_opt = np.array([np.cos(theta3_opt), np.sin(theta3_opt), 0])
```

```
# 投弹方向与无人机飞行方向一致
bomb_direction3_opt = drone_direction3_opt

# 导弹飞行方向
missile_direction = direction(M1, F)
v_missile = 300
total_missile_time = np.linalg.norm(M1 - F) / v_missile

# 时间参数
T = np.arange(0, total_missile_time, 0.01)
timeline = np.zeros(len(T))

count1, count2, count3 = 0, 0, 0
# 第一个烟幕弹
for i, t in enumerate(T):
    if count1 > 2000:
        break
    if t < A1_opt + B1_opt:
        continue

    # 计算烟幕弹位置
    n_jkt1 = np.array(FY1) + (A1_opt + B1_opt) * v_drone1_opt * bomb_direction1_opt -
0.5 * 9.8 * (
        B1_opt) ** 2 * np.array([0, 0, 1]) - 3 * (
                        t - A1_opt - B1_opt) * np.array([0, 0, 1])

    # 计算导弹位置
    M1_t = np.array(M1) + missile_direction * v_missile * t

    # 检验
    if test1(M1_t, n_jkt1):
        timeline[i] += 1
    count1 += 1
```

```
# 第二个烟幕弹
for i, t in enumerate(T):
    if count2 > 2000:
        break
    if t < A2_opt + B2_opt:
        continue

    # 计算烟幕弹位置
    n_jkt2 = np.array(FY2) + ( A2_opt + B2_opt) * v_drone2_opt * bomb_direction2_opt -
0.5 * 9.8 * (
        B2_opt) ** 2 * np.array(
        [0, 0, 1]) - 3 * (t -B2_opt - A2_opt) * np.array([0, 0, 1])

    # 计算导弹位置
    M1_t = np.array(M1) + missile_direction * v_missile * t

    # 检验
    if test1(M1_t, n_jkt2):
        timeline[i] += 1
    count2 += 1

# 第三个烟幕弹
for i, t in enumerate(T):
    if count3 > 2000:
        break
    if t < A3_opt + B3_opt:
        continue

    # 计算烟幕弹位置
    n_jkt3 = np.array(FY3) + ( A3_opt + B3_opt) * v_drone3_opt * bomb_direction3_opt -
0.5 * 9.8 * (
        B3_opt) ** 2 * np.array(
        [0, 0, 1]) - 3 * (t - B3_opt    - A3_opt) * np.array([0, 0, 1])

    # 计算导弹位置
```

```
    M1_t = np.array(M1) + missile_direction * v_missile * t

    # 检验
    if test1(M1_t, n_jkt3):
        timeline[i] += 1
    count3 += 1

# 确保值不超过 1
timeline = np.clip(timeline, 0, 1)

total_time = np.sum(timeline) * 0.01

plt.figure(figsize=(10, 6))
plt.plot(T, timeline)
plt.xlabel('时间 (s)')
plt.ylabel('是否在目标区域')
plt.title('优化后炸弹在目标区域的时间')
plt.grid(True)
plt.show()

print(f"优化后在目标区域总时间: {total_time} 秒")

# 绘制收敛曲线
plt.figure(figsize=(10, 6))
plt.plot(pso.fitness_history, 'b-', linewidth=2)
plt.title('PSO 算法收敛曲线', fontsize=14)
plt.xlabel('迭代次数', fontsize=12)
plt.ylabel('最佳适应度', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

问题五求解

```
# 目标函数 - 计算有效遮蔽时间
```

```
def objective_function(x):
    # 解包参数: (无人机速度, 无人机航向角,投放时间*3,起爆时间*3)*5
    (v_drone1, theta1, A11, B11 ,A12, B12, A13, B13,
     v_drone2, theta2, A21, B21 ,A22, B22, A23, B23,
     v_drone3, theta3, A31, B31 ,A32, B32, A33, B33,
     v_drone4, theta4, A41, B41 ,A42, B42, A43, B43,
     v_drone5, theta5, A51, B51 ,A52, B52, A53, B53) = x



    # 无人机 1 飞行方向 (水平于 xy 平面)
    drone_direction1 = np.array([np.cos(theta1), np.sin(theta1), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction1 = drone_direction1

    # 无人机 2 飞行方向 (水平于 xy 平面)
    drone_direction2 = np.array([np.cos(theta2), np.sin(theta2), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction2 = drone_direction2

    # 无人机 3 飞行方向 (水平于 xy 平面)
    drone_direction3 = np.array([np.cos(theta3), np.sin(theta3), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction3 = drone_direction3

    # 无人机 4 飞行方向 (水平于 xy 平面)
    drone_direction4 = np.array([np.cos(theta4), np.sin(theta4), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction4 = drone_direction4

    # 无人机 5 飞行方向 (水平于 xy 平面)
```

```
    drone_direction5 = np.array([np.cos(theta5), np.sin(theta5), 0])

    # 投弹方向与无人机飞行方向一致
    bomb_direction5 = drone_direction5

    v_missile = 300

    # 时间参数
    total_missile_time = np.linalg.norm(M1 - F) / v_missile
    T = np.arange(0, total_missile_time, 0.01)

    # 计算结果 - 直接使用列表，不使用 pandas Series
    timeline = np.zeros(len(T))

    # 导弹 1
    # 导弹飞行方向
    missile_direction1 = direction(M1, F)
    v_missile = 300   # 导弹速度 300m/s

    # 无人机 1
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A11 + B11:
            continue

        # 计算烟幕弹位置
        n_jkt11 = np.array(FY1) + (A11 + B11) * v_drone1 * bomb_direction1 - 0.5 * 9.8 *
(B11) ** 2 * np.array(
            [0, 0, 1]) - 3 * (
                                t - A11 - B11) * np.array([0, 0, 1])

        # 计算导弹位置
```

```
        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        # 检验
        if test1(M1_t, n_jkt11):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A11 + A12 + B12:
            continue

        # 计算烟幕弹位置
        n_jkt12 = np.array(FY1) + (A11 + A12 + B12) * v_drone1 * bomb_direction1 - 0.5
* 9.8 * (B12) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A11 - B12 - A12) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        # 检验
        if test1(M1_t, n_jkt12):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A11 + A12 + A13 + B13:
            continue

        # 计算烟幕弹位置
```

```
        n_jkt13 = np.array(FY1) + (A11 + A12 + A13 + B13) * v_drone1 * bomb_direction1 - 0.5 * 9.8 * (B13) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A11 - B13 - A12 - A13) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        # 检验
        if test1(M1_t, n_jkt13):
            timeline[i] += 1
        count3 += 1

    # 确保值不超过 1
    timeline = np.clip(timeline, 0, 1)

    total_time11 = np.sum(timeline) * 0.01   # 每个时间点代表 0.01 秒

    # 无人机 2
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A21 + B21:
            continue

        # 计算烟幕弹位置
        n_jkt21 = np.array(FY2) + (A21 + B21) * v_drone2 * bomb_direction2 - 0.5 * 9.8 * (B21) ** 2 * np.array(
            [0, 0, 1]) - 3 * (
                        t - A21 - B21) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction1 * v_missile * t
```

```
        # 检验
        if test1(M1_t, n_jkt21):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A21 + A22 + B22:
            continue

        # 计算烟幕弹位置
        n_jkt22 = np.array(FY2) + (A21 + A22 + B22) * v_drone2 * bomb_direction2 - 0.5
* 9.8 * (B22) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A21 - B22 - A22) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        # 检验
        if test1(M1_t, n_jkt22):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A21 + A22 + A23 + B23:
            continue

        # 计算烟幕弹位置
        n_jkt23 = np.array(FY2) + (A21 + A22 + A23 + B23) * v_drone2 *
bomb_direction2 - 0.5 * 9.8 * (
```

```
            B23) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A21 - B23 - A22 - A23) * np.array([0, 0, 1])

        # 计算导弹位置
        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        # 检验
        if test1(M1_t, n_jkt23):
            timeline[i] += 1
        count3 += 1

    # 确保值不超过 1
    timeline = np.clip(timeline, 0, 1)

    total_time12 = np.sum(timeline) * 0.01    # 每个时间点代表 0.01 秒

    # 无人机 3
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A31 + B31:
            continue

        n_jkt31 = np.array(FY3) + (A31 + B31) * v_drone3 * bomb_direction3 - 0.5 * 9.8 *
(B31) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A31 - B31) * np.array([0, 0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        if test1(M1_t, n_jkt31):
            timeline[i] += 1
        count1 += 1
```

```
    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A31 + A32 + B32:
            continue

        n_jkt32 = np.array(FY3) + (A31 + A32 + B32) * v_drone3 * bomb_direction3 - 0.5
* 9.8 * (B32) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A31 - B32 - A32) * np.array([0, 0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        if test1(M1_t, n_jkt32):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A31 + A32 + A33 + B33:
            continue

        n_jkt33 = np.array(FY3) + (A31 + A32 + A33 + B33) * v_drone3 *
bomb_direction3 - 0.5 * 9.8 * (
            B33) ** 2 * np.array([0, 0, 1]) - 3 * (t - A31 - B33 - A32 - A33) * np.array([0,
0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        if test1(M1_t, n_jkt33):
            timeline[i] += 1
        count3 += 1
```

```
    timeline = np.clip(timeline, 0, 1)
    total_time13 = np.sum(timeline) * 0.01

    # 无人机 4
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A41 + B41:
            continue

        n_jkt41 = np.array(FY4) + (A41 + B41) * v_drone4 * bomb_direction4 - 0.5 * 9.8 *
(B41) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A41 - B41) * np.array([0, 0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        if test1(M1_t, n_jkt41):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A41 + A42 + B42:
            continue

        n_jkt42 = np.array(FY4) + (A41 + A42 + B42) * v_drone4 * bomb_direction4 - 0.5
* 9.8 * (B42) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A41 - B42 - A42) * np.array([0, 0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t
```

```
        if test1(M1_t, n_jkt42):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A41 + A42 + A43 + B43:
            continue

        n_jkt43 = np.array(FY4) + (A41 + A42 + A43 + B43) * v_drone4 *
bomb_direction4 - 0.5 * 9.8 * (
            B43) ** 2 * np.array([0, 0, 1]) - 3 * (t - A41 - B43 - A42 - A43) * np.array([0,
0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        if test1(M1_t, n_jkt43):
            timeline[i] += 1
        count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time14 = np.sum(timeline) * 0.01

    # 无人机 5
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A51 + B51:
            continue

        n_jkt51 = np.array(FY5) + (A51 + B51) * v_drone5 * bomb_direction5 - 0.5 * 9.8 *
```

```
(B51) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - A51 - B51) * np.array([0, 0, 1])

            M1_t = np.array(M1) + missile_direction1 * v_missile * t

            if test1(M1_t, n_jkt51):
                timeline[i] += 1
            count1 += 1

        # 第二个烟幕弹
        for i, t in enumerate(T):
            if count2 > 2000:
                break
            if t < A51 + A52 + B52:
                continue

            n_jkt52 = np.array(FY5) + (A51 + A52 + B52) * v_drone5 * bomb_direction5 - 0.5
* 9.8 * (B52) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - A51 - B52 - A52) * np.array([0, 0, 1])

            M1_t = np.array(M1) + missile_direction1 * v_missile * t

            if test1(M1_t, n_jkt52):
                timeline[i] += 1
            count2 += 1

        # 第三个烟幕弹
        for i, t in enumerate(T):
            if count3 > 2000:
                break
            if t < A51 + A52 + A53 + B53:
                continue

            n_jkt53 = np.array(FY5) + (A51 + A52 + A53 + B53) * v_drone5 *
bomb_direction5 - 0.5 * 9.8 * (
```

```
                B53) ** 2 * np.array([0, 0, 1]) - 3 * (t - A51 - B53 - A52 - A53) * np.array([0,
0, 1])

        M1_t = np.array(M1) + missile_direction1 * v_missile * t

        if test1(M1_t, n_jkt53):
            timeline[i] += 1
        count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time15 = np.sum(timeline) * 0.01
    total_time1 = total_time15 + total_time14 + total_time13 + total_time12 + total_time11

    # 导弹 2
    # 导弹飞行方向
    missile_direction2 = direction(M2, F)
    v_missile = 300   # 导弹速度 300m/s

    # 初始化时间线数组
    timeline = np.zeros(len(T))

    # 无人机 1
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A11 + B11:
            continue

        # 计算烟幕弹位置
        n_jkt11 = np.array(FY1) + (A11 + B11) * v_drone1 * bomb_direction1 - 0.5 * 9.8 *
(B11) ** 2 * np.array(
            [0, 0, 1]) - 3 * (
                                t - A11 - B11) * np.array([0, 0, 1])
```

```
        # 计算导弹位置
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        # 检验
        if test1(M2_t, n_jkt11):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A11 + A12 + B12:
            continue

        # 计算烟幕弹位置
        n_jkt12 = np.array(FY1) + (A11 + A12 + B12) * v_drone1 * bomb_direction1 - 0.5
* 9.8 * (B12) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A11 - B12 - A12) * np.array([0, 0, 1])

        # 计算导弹位置
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        # 检验
        if test1(M2_t, n_jkt12):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A11 + A12 + A13 + B13:
            continue
```

```
        # 计算烟幕弹位置
        n_jkt13 = np.array(FY1) + (A11 + A12 + A13 + B13) * v_drone1 * bomb_direction1 - 0.5 * 9.8 * (
            B13) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A11 - B13 - A12 - A13) * np.array([0, 0, 1])

        # 计算导弹位置
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        # 检验
        if test1(M2_t, n_jkt13):
            timeline[i] += 1
        count3 += 1

    # 确保值不超过 1
    timeline = np.clip(timeline, 0, 1)

    total_time21 = np.sum(timeline) * 0.01   # 每个时间点代表 0.01 秒

    # 无人机 2
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A21 + B21:
            continue

        # 计算烟幕弹位置
        n_jkt21 = np.array(FY2) + (A21 + B21) * v_drone2 * bomb_direction2 - 0.5 * 9.8 * (B21) ** 2 * np.array(
            [0, 0, 1]) - 3 * (
                                t - A21 - B21) * np.array([0, 0, 1])
```

```
        # 计算导弹位置
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        # 检验
        if test1(M2_t, n_jkt21):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A21 + A22 + B22:
            continue

        # 计算烟幕弹位置
        n_jkt22 = np.array(FY2) + (A21 + A22 + B22) * v_drone2 * bomb_direction2 - 0.5
* 9.8 * (B22) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A21 - B22 - A22) * np.array([0, 0, 1])

        # 计算导弹位置
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        # 检验
        if test1(M2_t, n_jkt22):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A21 + A22 + A23 + B23:
            continue
```

```
        # 计算烟幕弹位置
        n_jkt23 = np.array(FY2) + (A21 + A22 + A23 + B23) * v_drone2 * bomb_direction2 - 0.5 * 9.8 * (
            B23) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A21 - B23 - A22 - A23) * np.array([0, 0, 1])

        # 计算导弹位置
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        # 检验
        if test1(M2_t, n_jkt23):
            timeline[i] += 1
        count3 += 1

    # 确保值不超过 1
    timeline = np.clip(timeline, 0, 1)

    total_time22 = np.sum(timeline) * 0.01   # 每个时间点代表 0.01 秒

    # 无人机 3
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A31 + B31:
            continue

        n_jkt31 = np.array(FY3) + (A31 + B31) * v_drone3 * bomb_direction3 - 0.5 * 9.8 * (B31) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A31 - B31) * np.array([0, 0, 1])

        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        if test1(M2_t, n_jkt31):
```

```
                timeline[i] += 1
            count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
            if count2 > 2000:
                break
            if t < A31 + A32 + B32:
                continue

            n_jkt32 = np.array(FY3) + (A31 + A32 + B32) * v_drone3 * bomb_direction3 - 0.5
* 9.8 * (B32) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - A31 - B32 - A32) * np.array([0, 0, 1])

            M2_t = np.array(M2) + missile_direction2 * v_missile * t

            if test1(M2_t, n_jkt32):
                timeline[i] += 1
            count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
            if count3 > 2000:
                break
            if t < A31 + A32 + A33 + B33:
                continue

            n_jkt33 = np.array(FY3) + (A31 + A32 + A33 + B33) * v_drone3 *
bomb_direction3 - 0.5 * 9.8 * (
                B33) ** 2 * np.array([0, 0, 1]) - 3 * (t - A31 - B33 - A32 - A33) * np.array([0,
0, 1])

            M2_t = np.array(M2) + missile_direction2 * v_missile * t

            if test1(M2_t, n_jkt33):
```

```
                timeline[i] += 1
            count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time23 = np.sum(timeline) * 0.01

    # 无人机 4
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A41 + B41:
            continue

        n_jkt41 = np.array(FY4) + (A41 + B41) * v_drone4 * bomb_direction4 - 0.5 * 9.8 *
(B41) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A41 - B41) * np.array([0, 0, 1])

        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        if test1(M2_t, n_jkt41):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A41 + A42 + B42:
            continue

        n_jkt42 = np.array(FY4) + (A41 + A42 + B42) * v_drone4 * bomb_direction4 - 0.5
* 9.8 * (B42) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A41 - B42 - A42) * np.array([0, 0, 1])
```

```
        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        if test1(M2_t, n_jkt42):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A41 + A42 + A43 + B43:
            continue

        n_jkt43 = np.array(FY4) + (A41 + A42 + A43 + B43) * v_drone4 * bomb_direction4 - 0.5 * 9.8 * (
            B43) ** 2 * np.array([0, 0, 1]) - 3 * (t - A41 - B43 - A42 - A43) * np.array([0, 0, 1])

        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        if test1(M2_t, n_jkt43):
            timeline[i] += 1
        count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time24 = np.sum(timeline) * 0.01

    # 无人机 5
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A51 + B51:
```

```
                continue

            n_jkt51 = np.array(FY5) + (A51 + B51) * v_drone5 * bomb_direction5 - 0.5 * 9.8 *
(B51) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - A51 - B51) * np.array([0, 0, 1])

            M2_t = np.array(M2) + missile_direction2 * v_missile * t

            if test1(M2_t, n_jkt51):
                timeline[i] += 1
            count1 += 1

        # 第二个烟幕弹
        for i, t in enumerate(T):
            if count2 > 2000:
                break
            if t < A51 + A52 + B52:
                continue

            n_jkt52 = np.array(FY5) + (A51 + A52 + B52) * v_drone5 * bomb_direction5 - 0.5
* 9.8 * (B52) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - A51 - B52 - A52) * np.array([0, 0, 1])

            M2_t = np.array(M2) + missile_direction2 * v_missile * t

            if test1(M2_t, n_jkt52):
                timeline[i] += 1
            count2 += 1

        # 第三个烟幕弹
        for i, t in enumerate(T):
            if count3 > 2000:
                break
            if t < A51 + A52 + A53 + B53:
                continue
```

```
        n_jkt53 = np.array(FY5) + (A51 + A52 + A53 + B53) * v_drone5 * bomb_direction5 - 0.5 * 9.8 * (
            B53) ** 2 * np.array([0, 0, 1]) - 3 * (t - A51 - B53 - A52 - A53) * np.array([0, 0, 1])

        M2_t = np.array(M2) + missile_direction2 * v_missile * t

        if test1(M2_t, n_jkt53):
            timeline[i] += 1
        count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time25 = np.sum(timeline) * 0.01
    total_time2 = total_time25 + total_time24 + total_time23 + total_time22 + total_time21

    # 导弹 3
    # 导弹飞行方向
    missile_direction3 = direction(M3, F)
    v_missile = 300   # 导弹速度 300m/s

    # 初始化时间线数组
    timeline = np.zeros(len(T))

    # 无人机 1
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A11 + B11:
            continue

        # 计算烟幕弹位置
        n_jkt11 = np.array(FY1) + (A11 + B11) * v_drone1 * bomb_direction1 - 0.5 * 9.8 *
```

```
(B11) ** 2 * np.array(
                [0, 0, 1]) - 3 * (
                                t - A11 - B11) * np.array([0, 0, 1])

        # 计算导弹位置
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        # 检验
        if test1(M3_t, n_jkt11):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A11 + A12 + B12:
            continue

        # 计算烟幕弹位置
        n_jkt12 = np.array(FY1) + (A11 + A12 + B12) * v_drone1 * bomb_direction1 - 0.5
* 9.8 * (B12) ** 2 * np.array(
                [0, 0, 1]) - 3 * (t - A11 - B12 - A12) * np.array([0, 0, 1])

        # 计算导弹位置
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        # 检验
        if test1(M3_t, n_jkt12):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
```

```
            break
        if t < A11 + A12 + A13 + B13:
            continue

        # 计算烟幕弹位置
        n_jkt13 = np.array(FY1) + (A11 + A12 + A13 + B13) * v_drone1 * bomb_direction1 - 0.5 * 9.8 * (
            B13) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A11 - B13 - A12 - A13) * np.array([0, 0, 1])

        # 计算导弹位置
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        # 检验
        if test1(M3_t, n_jkt13):
            timeline[i] += 1
        count3 += 1

    # 确保值不超过 1
    timeline = np.clip(timeline, 0, 1)

    total_time31 = np.sum(timeline) * 0.01    # 每个时间点代表 0.01 秒

    # 无人机 2
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A21 + B21:
            continue

        # 计算烟幕弹位置
        n_jkt21 = np.array(FY2) + (A21 + B21) * v_drone2 * bomb_direction2 - 0.5 * 9.8 * (B21) ** 2 * np.array(
```

```
                [0, 0, 1]) - 3 * (
                                t - A21 - B21) * np.array([0, 0, 1])

        # 计算导弹位置
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        # 检验
        if test1(M3_t, n_jkt21):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A21 + A22 + B22:
            continue

        # 计算烟幕弹位置
        n_jkt22 = np.array(FY2) + (A21 + A22 + B22) * v_drone2 * bomb_direction2 - 0.5
* 9.8 * (B22) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A21 - B22 - A22) * np.array([0, 0, 1])

        # 计算导弹位置
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        # 检验
        if test1(M3_t, n_jkt22):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
```

```
        if t < A21 + A22 + A23 + B23:
            continue

        # 计算烟幕弹位置
        n_jkt23 = np.array(FY2) + (A21 + A22 + A23 + B23) * v_drone2 * bomb_direction2 - 0.5 * 9.8 * (
            B23) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A21 - B23 - A22 - A23) * np.array([0, 0, 1])

        # 计算导弹位置
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        # 检验
        if test1(M3_t, n_jkt23):
            timeline[i] += 1
        count3 += 1

    # 确保值不超过 1
    timeline = np.clip(timeline, 0, 1)

    total_time32 = np.sum(timeline) * 0.01    # 每个时间点代表 0.01 秒

    # 无人机 3
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A31 + B31:
            continue

        n_jkt31 = np.array(FY3) + (A31 + B31) * v_drone3 * bomb_direction3 - 0.5 * 9.8 * (B31) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A31 - B31) * np.array([0, 0, 1])
```

```
        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt31):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A31 + A32 + B32:
            continue

        n_jkt32 = np.array(FY3) + (A31 + A32 + B32) * v_drone3 * bomb_direction3 - 0.5
* 9.8 * (B32) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A31 - B32 - A32) * np.array([0, 0, 1])

        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt32):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A31 + A32 + A33 + B33:
            continue

        n_jkt33 = np.array(FY3) + (A31 + A32 + A33 + B33) * v_drone3 *
bomb_direction3 - 0.5 * 9.8 * (
            B33) ** 2 * np.array([0, 0, 1]) - 3 * (t - A31 - B33 - A32 - A33) * np.array([0,
0, 1])
```

```
        M3_t = np.array(M3) + missile_direction3 * v_messile * t

        if test1(M3_t, n_jkt33):
            timeline[i] += 1
        count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time33 = np.sum(timeline) * 0.01

    # 无人机 4
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
        if count1 > 2000:
            break
        if t < A41 + B41:
            continue

        n_jkt41 = np.array(FY4) + (A41 + B41) * v_drone4 * bomb_direction4 - 0.5 * 9.8 *
(B41) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A41 - B41) * np.array([0, 0, 1])

        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt41):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A41 + A42 + B42:
            continue
```

```
        n_jkt42 = np.array(FY4) + (A41 + A42 + B42) * v_drone4 * bomb_direction4 - 0.5
* 9.8 * (B42) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A41 - B42 - A42) * np.array([0, 0, 1])

        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt42):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
            break
        if t < A41 + A42 + A43 + B43:
            continue

        n_jkt43 = np.array(FY4) + (A41 + A42 + A43 + B43) * v_drone4 *
bomb_direction4 - 0.5 * 9.8 * (
            B43) ** 2 * np.array([0, 0, 1]) - 3 * (t - A41 - B43 - A42 - A43) * np.array([0,
0, 1])

        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt43):
            timeline[i] += 1
        count3 += 1

    timeline = np.clip(timeline, 0, 1)
    total_time34 = np.sum(timeline) * 0.01

    # 无人机 5
    count1, count2, count3 = 0, 0, 0
    # 第一个烟幕弹
    for i, t in enumerate(T):
```

```
        if count1 > 2000:
            break
        if t < A51 + B51:
            continue

        n_jkt51 = np.array(FY5) + (A51 + B51) * v_drone5 * bomb_direction5 - 0.5 * 9.8 *
(B51) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A51 - B51) * np.array([0, 0, 1])

        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt51):
            timeline[i] += 1
        count1 += 1

    # 第二个烟幕弹
    for i, t in enumerate(T):
        if count2 > 2000:
            break
        if t < A51 + A52 + B52:
            continue

        n_jkt52 = np.array(FY5) + (A51 + A52 + B52) * v_drone5 * bomb_direction5 - 0.5
* 9.8 * (B52) ** 2 * np.array(
            [0, 0, 1]) - 3 * (t - A51 - B52 - A52) * np.array([0, 0, 1])

        M3_t = np.array(M3) + missile_direction3 * v_missile * t

        if test1(M3_t, n_jkt52):
            timeline[i] += 1
        count2 += 1

    # 第三个烟幕弹
    for i, t in enumerate(T):
        if count3 > 2000:
```

```
                break
            if t < A51 + A52 + A53 + B53:
                continue

            n_jkt53 = np.array(FY5) + (A51 + A52 + A53 + B53) * v_drone5 * bomb_direction5 - 0.5 * 9.8 * (
                B53) ** 2 * np.array([0, 0, 1]) - 3 * (t - A51 - B53 - A52 - A53) * np.array([0, 0, 1])

            M3_t = np.array(M3) + missile_direction3 * v_missile * t

            if test1(M3_t, n_jkt53):
                timeline[i] += 1
            count3 += 1

        timeline = np.clip(timeline, 0, 1)
        total_time35 = np.sum(timeline) * 0.01
        total_time3 = total_time35 + total_time34 + total_time33 + total_time32 + total_time31

        total_time = total_time3+total_time2+total_time1



        return -total_time

# PSO 算法实现
class PSO:
    def __init__(self, objective_func, bounds, num_particles=40, max_iter=260, w=0.9, c1=1.5, c2=1.5):
        self.objective_func = objective_func
        self.bounds = bounds
        self.num_particles = num_particles
        self.max_iter = max_iter
        self.w = w    # 惯性权重
        self.c1 = c1    # 个体学习因子
        self.c2 = c2    # 社会学习因子
```

```
        self.dim = len(bounds)

        # 初始化粒子位置和速度
        self.X = np.random.uniform(low=[b[0] for b in bounds],
                                   high=[b[1] for b in bounds],
                                   size=(num_particles, self.dim))
        self.V = np.random.uniform(-1, 1, (num_particles, self.dim))

        # 初始化个体最优位置和适应度
        self.P = self.X.copy()
        self.p_fitness = np.array([self.objective_func(x) for x in self.X])

        # 初始化全局最优位置和适应度
        self.g = self.P[np.argmin(self.p_fitness)]
        self.g_fitness = np.min(self.p_fitness)

        # 记录每次迭代的最佳适应度
        self.fitness_history = []

    def optimize(self):
        for iter in range(self.max_iter):
            # 更新惯性权重（线性递减）
            w = self.w * (1 - iter / self.max_iter)

            for i in range(self.num_particles):
                # 更新速度
                r1, r2 = np.random.rand(self.dim), np.random.rand(self.dim)
                self.V[i] = (w * self.V[i] +
                             self.c1 * r1 * (self.P[i] - self.X[i]) +
                             self.c2 * r2 * (self.g - self.X[i]))

                # 更新位置
                self.X[i] = self.X[i] + self.V[i]

                # 边界处理
```

```
                for j in range(self.dim):
                    if self.X[i, j] < self.bounds[j][0]:
                        self.X[i, j] = self.bounds[j][0]
                    elif self.X[i, j] > self.bounds[j][1]:
                        self.X[i, j] = self.bounds[j][1]

                # 计算新位置的适应度
                fitness = self.objective_func(self.X[i])

                # 更新个体最优
                if fitness < self.p_fitness[i]:
                    self.P[i] = self.X[i].copy()
                    self.p_fitness[i] = fitness

                # 更新全局最优
                if fitness < self.g_fitness:
                    self.g = self.X[i].copy()
                    self.g_fitness = fitness

            # 记录当前迭代的最佳适应度
            self.fitness_history.append(self.g_fitness)

            # 打印当前迭代信息
            if (iter + 1) % 10 == 0:
                print(f"Iteration {iter + 1}/{self.max_iter}, Best Fitness: {-self.g_fitness:.4f}")

        return self.g, self.g_fitness


# 定义参数边界
# (无人机速度, 无人机航向角,投放时间*3,起爆时间*3)
bounds = [(70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7), (1, 7), (0.01, 7), (2, 7), (0.01, 7),
          (70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7), (1, 7), (0.01, 7), (2, 7), (0.01, 7),
          (70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7), (1, 7), (0.01, 7), (2, 7), (0.01, 7),
```

```
                (70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7), (1, 7), (0.01, 7), (2, 7), (0.01, 7),
                (70, 140), (0, 2 * np.pi), (0.01, 7), (0.01, 7), (1, 7), (0.01, 7), (2, 7), (0.01, 7)]

# 使用 PSO 算法进行优化
print("开始 PSO 优化...")
pso = PSO(objective_function, bounds, num_particles=60, max_iter=100)
xopt, fopt = pso.optimize()

print("PSO 优化结果:")
print(
    f"最佳参数: ")
print(f"最大有效遮蔽时间: {-fopt} 秒")

# 使用最佳参数计算并可视化结果
# 输出最优参数
print("PSO 优化结果:")
print("最佳参数:")
param_names = [
    "v_drone1", "theta1", "A11", "B11", "A12", "B12", "A13", "B13",
    "v_drone2", "theta2", "A21", "B21", "A22", "B22", "A23", "B23",
    "v_drone3", "theta3", "A31", "B31", "A32", "B32", "A33", "B33",
    "v_drone4", "theta4", "A41", "B41", "A42", "B42", "A43", "B43",
    "v_drone5", "theta5", "A51", "B51", "A52", "B52", "A53", "B53"
]

for i, (name, value) in enumerate(zip(param_names, xopt)):
    print(f"{name}: {value:.4f}")

print(f"最大有效遮蔽时间: {-fopt:.4f} 秒")

# 绘制收敛曲线
plt.figure(figsize=(10, 6))
plt.plot(pso.fitness_history, 'b-', linewidth=2)
plt.title('PSO 算法收敛曲线', fontsize=14)
plt.xlabel('迭代次数', fontsize=12)
```

```
plt.ylabel('最佳适应度', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```